

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

```
├─ assets/
│   └─ index-BM6zjEMh.js (64546 B / 8946440ab22e)
└─ index-Cb95KwZL.css (14408 B / 2a57b7ea8268)
└─ index.html (7534 B / bc726d2a0eb1)
```

==== assets/index-BM6zjEMh.js (64546 B / 8946440ab22e) ====

```
1 | const GAME_MODE_IDS = ['marathon', 'fortyLines', 'blitz'],
2 | GAME_MODE_LABELS = { marathon: 'Marathon', fortyLines: '40 Lines', blitz: 'BLITZ' },
3 | GAME_MODE_RECORD_METRICS = { marathon: 'score', fortyLines: 'time', blitz: 'score' },
4 | isGameModeId = (value) => 'string' === typeof value && GAME_MODE_IDS.includes(value),
5 | JLISTZ_90_KICKS = {
6 |   '0>1': [
7 |     { x: 0, y: 0 },
8 |     { x: -1, y: 0 },
9 |     { x: -1, y: 1 },
10 |    { x: 0, y: -2 },
11 |    { x: -1, y: -2 },
12 |   ],
13 |   '1>0': [
14 |     { x: 0, y: 0 },
15 |     { x: 1, y: 0 },
16 |     { x: 1, y: -1 },
17 |     { x: 0, y: 2 },
18 |     { x: 1, y: 2 },
19 |   ],
20 |   '1>2': [
21 |     { x: 0, y: 0 },
22 |     { x: 1, y: 0 },
23 |     { x: 1, y: -1 },
24 |     { x: 0, y: 2 },
25 |     { x: 1, y: 2 },
26 |   ],
27 |   '2>1': [
28 |     { x: 0, y: 0 },
29 |     { x: -1, y: 0 },
30 |     { x: -1, y: 1 },
31 |     { x: 0, y: -2 },
32 |     { x: -1, y: -2 },
33 |   ],
34 |   '2>3': [
35 |     { x: 0, y: 0 },
36 |     { x: 1, y: 0 },
37 |     { x: 1, y: 1 },
38 |     { x: 0, y: -2 },
39 |     { x: 1, y: -2 },
40 |   ],
41 |   '3>2': [
42 |     { x: 0, y: 0 },
43 |     { x: -1, y: 0 },
44 |     { x: -1, y: -1 },
45 |     { x: 0, y: 2 },
46 |     { x: -1, y: 2 },
47 |   ],
48 |   '3>0': [
49 |     { x: 0, y: 0 },
50 |     { x: -1, y: 0 },
51 |     { x: -1, y: -1 },
52 |     { x: 0, y: 2 },
53 |     { x: -1, y: 2 },
54 |   ],
55 |   '0>3': [
56 |     { x: 0, y: 0 },
57 |     { x: 1, y: 0 },
58 |     { x: 1, y: 1 },
59 |     { x: 0, y: -2 },
60 |     { x: 1, y: -2 },
```

```

61 |     ],
62 | },
63 | I_90_KICKS = {
64 |     '0>1': [
65 |         {x: 0, y: 0},
66 |         {x: -2, y: 0},
67 |         {x: 1, y: 0},
68 |         {x: -2, y: -1},
69 |         {x: 1, y: 2},
70 |     ],
71 |     '1>0': [
72 |         {x: 0, y: 0},
73 |         {x: 2, y: 0},
74 |         {x: -1, y: 0},
75 |         {x: 2, y: 1},
76 |         {x: -1, y: -2},
77 |     ],
78 |     '1>2': [
79 |         {x: 0, y: 0},
80 |         {x: -1, y: 0},
81 |         {x: 2, y: 0},
82 |         {x: -1, y: 2},
83 |         {x: 2, y: -1},
84 |     ],
85 |     '2>1': [
86 |         {x: 0, y: 0},
87 |         {x: 1, y: 0},
88 |         {x: -2, y: 0},
89 |         {x: 1, y: -2},
90 |         {x: -2, y: 1},
91 |     ],
92 |     '2>3': [
93 |         {x: 0, y: 0},
94 |         {x: 2, y: 0},
95 |         {x: -1, y: 0},
96 |         {x: 2, y: 1},
97 |         {x: -1, y: -2},
98 |     ],
99 |     '3>2': [
100 |         {x: 0, y: 0},
101 |         {x: -2, y: 0},
102 |         {x: 1, y: 0},
103 |         {x: -2, y: -1},
104 |         {x: 1, y: 2},
105 |     ],
106 |     '3>0': [
107 |         {x: 0, y: 0},
108 |         {x: 1, y: 0},
109 |         {x: -2, y: 0},
110 |         {x: 1, y: -2},
111 |         {x: -2, y: 1},
112 |     ],
113 |     '0>3': [
114 |         {x: 0, y: 0},
115 |         {x: -1, y: 0},
116 |         {x: 2, y: 0},
117 |         {x: -1, y: 2},
118 |         {x: 2, y: -1},
119 |     ],
120 | },
121 | JLSTZ_180_KICKS = {
122 |     '0>2': [
123 |         {x: 0, y: 0},
124 |         {x: 1, y: 0},
125 |         {x: -1, y: 0},
126 |         {x: 0, y: 1},
127 |         {x: 1, y: 1},
128 |         {x: -1, y: 1},
129 |         {x: 0, y: -1},
130 |         {x: 2, y: 0},
131 |         {x: -2, y: 0},

```

```

132 |     ],
133 |     '2>0': [
134 |         {x: 0, y: 0},
135 |         {x: 1, y: 0},
136 |         {x: -1, y: 0},
137 |         {x: 0, y: -1},
138 |         {x: 1, y: -1},
139 |         {x: -1, y: -1},
140 |         {x: 0, y: 1},
141 |         {x: 2, y: 0},
142 |         {x: -2, y: 0},
143 |     ],
144 |     '1>3': [
145 |         {x: 0, y: 0},
146 |         {x: 0, y: 1},
147 |         {x: 1, y: 0},
148 |         {x: -1, y: 0},
149 |         {x: 2, y: 0},
150 |         {x: -2, y: 0},
151 |     ],
152 |     '3>1': [
153 |         {x: 0, y: 0},
154 |         {x: 0, y: 1},
155 |         {x: 1, y: 0},
156 |         {x: -1, y: 0},
157 |         {x: 2, y: 0},
158 |         {x: -2, y: 0},
159 |         {x: 1, y: 1},
160 |         {x: -1, y: 1},
161 |     ],
162 | },
163 | I_180_KICKS = {
164 |     '0>2': [
165 |         {x: 0, y: 0},
166 |         {x: -1, y: 0},
167 |         {x: 2, y: 0},
168 |         {x: -1, y: 2},
169 |         {x: 2, y: -1},
170 |         {x: 0, y: 1},
171 |         {x: 0, y: -1},
172 |     ],
173 |     '2>0': [
174 |         {x: 0, y: 0},
175 |         {x: 1, y: 0},
176 |         {x: -2, y: 0},
177 |         {x: 1, y: -2},
178 |         {x: -2, y: 1},
179 |         {x: 0, y: 1},
180 |         {x: 0, y: -1},
181 |     ],
182 |     '1>3': [
183 |         {x: 0, y: 0},
184 |         {x: 0, y: 1},
185 |         {x: 0, y: -1},
186 |         {x: 1, y: 0},
187 |         {x: -2, y: 0},
188 |         {x: 1, y: 2},
189 |         {x: -2, y: -1},
190 |     ],
191 |     '3>1': [
192 |         {x: 0, y: 0},
193 |         {x: 0, y: 1},
194 |         {x: 0, y: -1},
195 |         {x: -1, y: 0},
196 |         {x: 2, y: 0},
197 |         {x: -1, y: -2},
198 |         {x: 2, y: 1},
199 |     ],
200 | },
201 | O_KICK = [{x: 0, y: 0}],
202 | CW_ROTATION = [1, 2, 3, 0],

```

```

203 | ·CCW_ROTATION = [3, 0, 1, 2],
204 | ·DOUBLE_ROTATION = [2, 3, 0, 1],
205 | ·getTargetRotation = (rotation, direction) =>
206 |   ·'cw' === direction
207 |   ·? CW_ROTATION[rotation]
208 |   ·:'ccw' === direction
209 |   ·? CCW_ROTATION[rotation]
210 |   ·: DOUBLE_ROTATION[rotation],
211 | ·LINE_CLEAR_SCORE = { 0: 0, 1: 100, 2: 300, 3: 500, 4: 800 },
212 | ·TSPIN_SCORE = { 0: 400, 1: 800, 2: 1200, 3: 1600 },
213 | ·TSPIN_MINI_SCORE = { 0: 100, 1: 200, 2: 400 },
214 | ·LINE_CLEAR_ATTACK = { 0: 0, 1: 0, 2: 1, 3: 2, 4: 4 },
215 | ·TSPIN_ATTACK = { 0: 0, 1: 2, 2: 4, 3: 6 },
216 | ·TSPIN_MINI_ATTACK = { 0: 0, 1: 0, 2: 1 },
217 | ·blitzLinesRequiredForLevel = (level) => {
218 |   ·const normalizedLevel = Math.max(1, Math.floor(level));
219 |   ·return normalizedLevel ≤ 10 ? 2 * normalizedLevel + 1 : 2 * normalizedLevel + 2;
220 | },
221 | ·blitzLevelForLines = (lines) => {
222 |   ·let level = 1,
223 |   ·remainingLines = Math.max(0, Math.floor(lines));
224 |   ·for (; remainingLines ≥ blitzLinesRequiredForLevel(level);)
225 |     ·((remainingLines -= blitzLinesRequiredForLevel(level)), (level += 1));
226 |   ·return level;
227 | },
228 | ·isDifficultClear = (lines, tspin) => !(lines ≤ 0 || (4 !== lines && 'none' === tspin)),
229 | ·keepsBackToBack = (lines, tspin, perfectClear = !1) =>
230 |   ·!!perfectClear || 0 === lines || isDifficultClear(lines, tspin),
231 | ·DAS_RANGE = { min: 0, max: 300 },
232 | ·ARR_RANGE = { min: 0, max: 100 },
233 | ·SDF_RANGE = { min: 1, max: 40 },
234 | ·DEFAULT_HANDLING = { dasMs: 165, arrMs: 0, sdfG: 20 },
235 | ·clamp$1 = (value, min, max) => Math.min(Math.max(value, min), max),
236 | ·isRecord = (value) => 'object' === typeof value && null !== value && !Array.isArray(value),
237 | ·normalizeInteger = (value, min) => {
238 |   ·const parsed = 'number' === typeof value ? value : Number(value);
239 |   ·return Number.isFinite(parsed) ? Math.max(min, Math.floor(parsed)) : null;
240 | },
241 | ·readJson = (key) => {
242 |   ·try {
243 |     ·const raw = globalThis.localStorage?.getItem(key);
244 |     ·return raw ? JSON.parse(raw) : null;
245 |   } catch {
246 |     ·return null;
247 |   }
248 | },
249 | ·writeJson = (key, value) => {
250 |   ·try {
251 |     ·globalThis.localStorage?.setItem(key, JSON.stringify(value));
252 |   } catch {}
253 | },
254 | ·normalizeMetricForMode = (mode, value) => {
255 |   ·const normalized = normalizeInteger(value, 0);
256 |   ·return null === normalized || ('time' === GAME_MODE_RECORD_METRICS[mode] && normalized ≤ 0)
257 |     ? null
258 |     : normalized;
259 | },
260 | ·compareLeaderboardEntries = (mode) => (a, b) =>
261 |   ·'time' === GAME_MODE_RECORD_METRICS[mode]
262 |   ·? a.elapsedMs - b.elapsedMs || b.score - a.score || a.createdAt - b.createdAt
263 |   ·: b.score - a.score || a.createdAt - b.createdAt,
264 | ·normalizeLeaderboardEntries = (mode, entries) =>
265 |   ·Array.isArray(entries)
266 |   ·? entries
267 |   ·: entries.map((entry) =>
268 |     ·((value, fallbackMode) => {
269 |       ·if (!isRecord(value)) return null;
270 |       ·const mode = isGameModeId(value.mode) ? value.mode : fallbackMode,
271 |       ·id = 'string' === typeof value.id ? value.id.trim() : '',
272 |       ·score = normalizeInteger(value.score, 0),
273 |       ·lines = normalizeInteger(value.lines, 0),

```

```

274 |         elapsedMs = normalizeInteger(value.elapsedMs, 0) ?? 0,
275 |         createdAt = normalizeInteger(value.createdAt, 0);
276 |         return id && null !== score && null !== lines && null !== createdAt
277 |         ? 'time' === GAME_MODE_RECORD_METRICS[mode] && elapsedMs ≤ 0
278 |         : null
279 |         : {
280 |             id: id,
281 |             mode: mode,
282 |             name:
283 |                 ((name = value.name),
284 |                 (('string' === typeof name ? name.trim() : '')) || 'PLAYER').slice(0, 12)),
285 |             score: score,
286 |             lines: lines,
287 |             elapsedMs: elapsedMs,
288 |             createdAt: createdAt,
289 |         };
290 |         : null;
291 |         var name;
292 |     })(entry, mode),
293 | )
294 |     .filter((entry) => null !== entry && entry.mode === mode)
295 |     .sort(compareLeaderboardEntries(mode))
296 |     .slice(0, 20)
297 |     : [],
298 | loadLegacyHighScore = () => {
299 |     try {
300 |         const raw = globalThis.localStorage?.getItem('tetris_high_score_v1'),
301 |             parsed = raw ? Number(raw) : 0;
302 |         return Number.isFinite(parsed) ? Math.max(0, Math.floor(parsed)) : 0;
303 |     } catch {
304 |         return 0;
305 |     }
306 | },
307 | loadBestMetrics = () =>
308 |     ((value) => {
309 |         const metrics = { marathon: null, fortyLines: null, blitz: null };
310 |         if (!isRecord(value)) return ((metrics.marathon = loadLegacyHighScore()), metrics);
311 |         for (const mode of GAME_MODE_IDS) metrics[mode] = normalizeMetricForMode(mode, value[mode]);
312 |         return (null === metrics.marathon && (metrics.marathon = loadLegacyHighScore()), metrics);
313 |     })(readJson('tetris_best_records_by_mode_v2')),
314 | loadLeaderboards = () => {
315 |     const parsed = readJson('tetris_leaderboards_by_mode_v2'),
316 |         leaderboards = { marathon: [], fortyLines: [], blitz: [] };
317 |     if (!isRecord(parsed))
318 |         return (
319 |             (leaderboards.marathon = () => {
320 |                 const parsed = readJson('tetris_leaderboard_v1');
321 |                 return normalizeLeaderboardEntries('marathon', parsed);
322 |             })(),
323 |             leaderboards
324 |         );
325 |     for (const mode of GAME_MODE_IDS)
326 |         leaderboards[mode] = normalizeLeaderboardEntries(mode, parsed[mode]);
327 |     return leaderboards;
328 | },
329 | normalizeHandling = (partial) => ({
330 |     dasMs: clamp$1(
331 |         Number.isFinite(partial.dasMs)
332 |         ? Math.round(partial.dasMs ?? DEFAULT_HANDLING.dasMs)
333 |         : DEFAULT_HANDLING.dasMs,
334 |         DAS_RANGE.min,
335 |         DAS_RANGE.max,
336 |     ),
337 |     arrMs: clamp$1(
338 |         Number.isFinite(partial.arrMs)
339 |         ? Math.round(partial.arrMs ?? DEFAULT_HANDLING.arrMs)
340 |         : DEFAULT_HANDLING.arrMs,
341 |         ARR_RANGE.min,
342 |         ARR_RANGE.max,
343 |     ),
344 |     sdfg: clamp$1(

```

```

345 |     ·Number.isFinite(partial.sdfG)
346 |     ·? Number(partial.sdfG ?? DEFAULT_HANDLING.sdfG)
347 |     ·: DEFAULT_HANDLING.sdfG,
348 |     ·SDF_RANGE.min,
349 |     ·SDF_RANGE.max,
350 |     ·),
351 |   ·}),
352 |   ·loadBestMetric := (mode) => {
353 |     ·const metric := loadBestMetrics()[mode];
354 |     ·return 'score' === GAME_MODE_RECORD_METRICS[mode] ? (metric ?? 0) : metric;
355 |   ·},
356 |   ·loadLeaderboard := (mode := 'marathon') => loadLeaderboards()[mode],
357 |   ·handlingRanges_dasMs := DAS_RANGE,
358 |   ·handlingRanges_arrMs := ARR_RANGE,
359 |   ·handlingRanges_sdfG := SDF_RANGE,
360 |   ·BLITZ_GRAVITY_BY_LEVEL_G := [
361 |     ·0, 0.0167, 0.0259, 0.0412, 0.067, 0.111, 0.189, 0.33, 0.588, 1.08, 2.01, 3.87, 7.62, 15.4, 20,
362 |   ·],
363 |   ·PIECE_ORDER := ['I', 'O', 'T', 'S', 'Z', 'J', 'L'],
364 |   ·createEmptyRow := () => Array.from({ length: 10 }, () => null),
365 |   ·createBoardFromRows := (rows) => {
366 |     ·const board := Array.from({ length: 40 }, () => createEmptyRow());
367 |     ·if (!rows) return board;
368 |     ·for (let y := 0; y < Math.min(rows.length, 40); y += 1)
369 |       ·for (let x := 0; x < Math.min(rows[y].length, 10); x += 1) board[y][x] := rows[y][x] ?? null;
370 |     ·return board;
371 |   ·},
372 |   ·createShuffledBag := (random) => {
373 |     ·const bag := [... PIECE_ORDER];
374 |     ·for (let index := bag.length - 1; index > 0; index -= 1) {
375 |       ·const swapIndex := Math.floor(random() * (index + 1)),
376 |       ·current := bag[index];
377 |       ·((bag[index] := bag[swapIndex]), (bag[swapIndex] := current));
378 |     ·}
379 |     ·return bag;
380 |   ·},
381 |   ·PIECE_CELLS := {
382 |     ·I: [
383 |       ·[
384 |         ·{ x: 0, y: 1 },
385 |         ·{ x: 1, y: 1 },
386 |         ·{ x: 2, y: 1 },
387 |         ·{ x: 3, y: 1 },
388 |       ·],
389 |     ·[
390 |       ·{ x: 2, y: 0 },
391 |       ·{ x: 2, y: 1 },
392 |       ·{ x: 2, y: 2 },
393 |       ·{ x: 2, y: 3 },
394 |     ·],
395 |     ·[
396 |       ·{ x: 0, y: 2 },
397 |       ·{ x: 1, y: 2 },
398 |       ·{ x: 2, y: 2 },
399 |       ·{ x: 3, y: 2 },
400 |     ·],
401 |     ·[
402 |       ·{ x: 1, y: 0 },
403 |       ·{ x: 1, y: 1 },
404 |       ·{ x: 1, y: 2 },
405 |       ·{ x: 1, y: 3 },
406 |     ·],
407 |   ·],
408 |   ·O: [
409 |     ·[
410 |       ·{ x: 1, y: 0 },
411 |       ·{ x: 2, y: 0 },
412 |       ·{ x: 1, y: 1 },
413 |       ·{ x: 2, y: 1 },
414 |     ·],
415 |   ·[

```

```

416 | . | . | . | { x: 1, y: 0 },
417 | . | . | . | { x: 2, y: 0 },
418 | . | . | . | { x: 1, y: 1 },
419 | . | . | . | { x: 2, y: 1 },
420 | . | . | . | ],
421 | . | . | . | [
422 | . | . | . | { x: 1, y: 0 },
423 | . | . | . | { x: 2, y: 0 },
424 | . | . | . | { x: 1, y: 1 },
425 | . | . | . | { x: 2, y: 1 },
426 | . | . | . | ],
427 | . | . | . | [
428 | . | . | . | { x: 1, y: 0 },
429 | . | . | . | { x: 2, y: 0 },
430 | . | . | . | { x: 1, y: 1 },
431 | . | . | . | { x: 2, y: 1 },
432 | . | . | . | ],
433 | . | . | . | ],
434 | . | . | . | T: [
435 | . | . | . | [
436 | . | . | . | { x: 1, y: 0 },
437 | . | . | . | { x: 0, y: 1 },
438 | . | . | . | { x: 1, y: 1 },
439 | . | . | . | { x: 2, y: 1 },
440 | . | . | . | ],
441 | . | . | . | [
442 | . | . | . | { x: 1, y: 0 },
443 | . | . | . | { x: 1, y: 1 },
444 | . | . | . | { x: 2, y: 1 },
445 | . | . | . | { x: 1, y: 2 },
446 | . | . | . | ],
447 | . | . | . | [
448 | . | . | . | { x: 0, y: 1 },
449 | . | . | . | { x: 1, y: 1 },
450 | . | . | . | { x: 2, y: 1 },
451 | . | . | . | { x: 1, y: 2 },
452 | . | . | . | ],
453 | . | . | . | [
454 | . | . | . | { x: 1, y: 0 },
455 | . | . | . | { x: 0, y: 1 },
456 | . | . | . | { x: 1, y: 1 },
457 | . | . | . | { x: 1, y: 2 },
458 | . | . | . | ],
459 | . | . | . | ],
460 | . | . | . | S: [
461 | . | . | . | [
462 | . | . | . | { x: 1, y: 0 },
463 | . | . | . | { x: 2, y: 0 },
464 | . | . | . | { x: 0, y: 1 },
465 | . | . | . | { x: 1, y: 1 },
466 | . | . | . | ],
467 | . | . | . | [
468 | . | . | . | { x: 1, y: 0 },
469 | . | . | . | { x: 1, y: 1 },
470 | . | . | . | { x: 2, y: 1 },
471 | . | . | . | { x: 2, y: 2 },
472 | . | . | . | ],
473 | . | . | . | [
474 | . | . | . | { x: 1, y: 1 },
475 | . | . | . | { x: 2, y: 1 },
476 | . | . | . | { x: 0, y: 2 },
477 | . | . | . | { x: 1, y: 2 },
478 | . | . | . | ],
479 | . | . | . | [
480 | . | . | . | { x: 0, y: 0 },
481 | . | . | . | { x: 0, y: 1 },
482 | . | . | . | { x: 1, y: 1 },
483 | . | . | . | { x: 1, y: 2 },
484 | . | . | . | ],
485 | . | . | . | ],
486 | . | . | . | Z: [

```

```

487 | | | | '['
488 | | | | '{ x: 0, y: 0 }',
489 | | | | '{ x: 1, y: 0 }',
490 | | | | '{ x: 1, y: 1 }',
491 | | | | '{ x: 2, y: 1 }',
492 | | | | ']',
493 | | | | '['
494 | | | | '{ x: 2, y: 0 }',
495 | | | | '{ x: 1, y: 1 }',
496 | | | | '{ x: 2, y: 1 }',
497 | | | | '{ x: 1, y: 2 }',
498 | | | | ']',
499 | | | | '['
500 | | | | '{ x: 0, y: 1 }',
501 | | | | '{ x: 1, y: 1 }',
502 | | | | '{ x: 1, y: 2 }',
503 | | | | '{ x: 2, y: 2 }',
504 | | | | ']',
505 | | | | '['
506 | | | | '{ x: 1, y: 0 }',
507 | | | | '{ x: 0, y: 1 }',
508 | | | | '{ x: 1, y: 1 }',
509 | | | | '{ x: 0, y: 2 }',
510 | | | | ']',
511 | | | | ']',
512 | | | | 'J: ['
513 | | | | '['
514 | | | | '{ x: 0, y: 0 }',
515 | | | | '{ x: 0, y: 1 }',
516 | | | | '{ x: 1, y: 1 }',
517 | | | | '{ x: 2, y: 1 }',
518 | | | | ']',
519 | | | | '['
520 | | | | '{ x: 1, y: 0 }',
521 | | | | '{ x: 2, y: 0 }',
522 | | | | '{ x: 1, y: 1 }',
523 | | | | '{ x: 1, y: 2 }',
524 | | | | ']',
525 | | | | '['
526 | | | | '{ x: 0, y: 1 }',
527 | | | | '{ x: 1, y: 1 }',
528 | | | | '{ x: 2, y: 1 }',
529 | | | | '{ x: 2, y: 2 }',
530 | | | | ']',
531 | | | | '['
532 | | | | '{ x: 1, y: 0 }',
533 | | | | '{ x: 1, y: 1 }',
534 | | | | '{ x: 0, y: 2 }',
535 | | | | '{ x: 1, y: 2 }',
536 | | | | ']',
537 | | | | ']',
538 | | | | 'L: ['
539 | | | | '['
540 | | | | '{ x: 2, y: 0 }',
541 | | | | '{ x: 0, y: 1 }',
542 | | | | '{ x: 1, y: 1 }',
543 | | | | '{ x: 2, y: 1 }',
544 | | | | ']',
545 | | | | '['
546 | | | | '{ x: 1, y: 0 }',
547 | | | | '{ x: 1, y: 1 }',
548 | | | | '{ x: 1, y: 2 }',
549 | | | | '{ x: 2, y: 2 }',
550 | | | | ']',
551 | | | | '['
552 | | | | '{ x: 0, y: 1 }',
553 | | | | '{ x: 1, y: 1 }',
554 | | | | '{ x: 2, y: 1 }',
555 | | | | '{ x: 0, y: 2 }',
556 | | | | ']',
557 | | | | '['

```



```

558 |     { x: 0, y: 0 },
559 |     { x: 1, y: 0 },
560 |     { x: 1, y: 1 },
561 |     { x: 1, y: 2 },
562 |   ],
563 | ],
564 | },
565 | gravityForState = (state) =>
566 |   'blitz' === state.mode
567 |   ? ((level) => {
568 |     const normalizedLevel = Math.max(1, Math.floor(level));
569 |     return BLITZ_GRAVITY_BY_LEVEL_G[normalizedLevel] ?? 20;
570 |   })(blitzLevelForLines(state.lines))
571 |   : ((elapsedMs) => {
572 |     const gravity =
573 |       0.02 + ((Math.max(0, elapsedMs - 120000.00000000001) / 1e3) * 0.0035) / 10;
574 |     return Math.min(gravity, 20);
575 |   })(state.elapsedMs),
576 | pieceCellsFor = (pieceType, rotation) => PIECE_CELLS[pieceType][rotation],
577 | getAbsoluteCells = (piece) =>
578 |   pieceCellsFor(piece.type, piece.rotation).map((cell) => ({
579 |     x: piece.x + cell.x,
580 |     y: piece.y + cell.y,
581 |   })),
582 | createActivePiece = (pieceType) => ({
583 |   type: pieceType,
584 |   x: 3,
585 |   y: 18,
586 |   rotation: 0,
587 |   lockDelayMs: 0,
588 |   lockResets: 0,
589 |   lastAction: 'none',
590 |   lastKickIndex: null,
591 |   softDropCells: 0,
592 |   hardDropCells: 0,
593 | }),
594 | isCellBlocked = (board, x, y) =>
595 |   !((x, y) => x < 0 || x ≥ 10 || y ≥ 40)(x, y) || !(y < 0) && null !== board[y][x]),
596 | collides = (board, piece) =>
597 |   getAbsoluteCells(piece).some((cell) => isCellBlocked(board, cell.x, cell.y)),
598 | isGrounded = (board, piece) => {
599 |   const moved = { ... piece, y: piece.y + 1 };
600 |   return collides(board, moved);
601 | },
602 | normalizeVisiblePoint = (point) => {
603 |   const visibleY = point.y - 20;
604 |   return visibleY < 0 || visibleY ≥ 20 ? null : { x: point.x, y: visibleY };
605 | },
606 | computeGhostCells = (board, active) => {
607 |   if (!active) return [];
608 |   const ghost = { ... active };
609 |   for (; !collides(board, { ... ghost, y: ghost.y + 1 }); ) ghost.y += 1;
610 |   return getAbsoluteCells(ghost)
611 |     .map(normalizeVisiblePoint)
612 |     .filter((point) => null !== point);
613 | },
614 | createInitialState = (random, handling = DEFAULT_HANDLING, initialBoard, mode = 'marathon') => ({
615 |   mode: mode,
616 |   status: 'ready',
617 |   board: createBoardFromRows(initialBoard),
618 |   active: null,
619 |   hold: null,
620 |   canHold: !0,
621 |   queue: [],
622 |   bag: [],
623 |   score: 0,
624 |   attackSent: 0,
625 |   lines: 0,
626 |   elapsedMs: 0,
627 |   combo: -1,
628 |   b2bChain: 0,

```

```

629 |     surgeCharge: 0,
630 |     clearFeedbackId: 0,
631 |     lastClearFeedback: null,
632 |     handling: handling,
633 |     softDropActive: !1,
634 |     gravityCarry: 0,
635 |     random: random,
636 |   })),
637 |   HANDLED_CODES := new Set([
638 |     'ArrowLeft',
639 |     'ArrowRight',
640 |     'ArrowDown',
641 |     'ArrowUp',
642 |     'Space',
643 |     'KeyZ',
644 |     'KeyX',
645 |     'KeyA',
646 |     'KeyC',
647 |     'ShiftLeft',
648 |     'ShiftRight',
649 |     'KeyP',
650 |     'KeyR',
651 |     'Enter',
652 |   ]),
653 |   PREVIEW_CELLS := {
654 |     I: [
655 |       {x: 0, y: 1},
656 |       {x: 1, y: 1},
657 |       {x: 2, y: 1},
658 |       {x: 3, y: 1},
659 |     ],
660 |     O: [
661 |       {x: 1, y: 1},
662 |       {x: 2, y: 1},
663 |       {x: 1, y: 2},
664 |       {x: 2, y: 2},
665 |     ],
666 |     T: [
667 |       {x: 1, y: 0},
668 |       {x: 0, y: 1},
669 |       {x: 1, y: 1},
670 |       {x: 2, y: 1},
671 |     ],
672 |     S: [
673 |       {x: 1, y: 0},
674 |       {x: 2, y: 0},
675 |       {x: 0, y: 1},
676 |       {x: 1, y: 1},
677 |     ],
678 |     Z: [
679 |       {x: 0, y: 0},
680 |       {x: 1, y: 0},
681 |       {x: 1, y: 1},
682 |       {x: 2, y: 1},
683 |     ],
684 |     J: [
685 |       {x: 0, y: 0},
686 |       {x: 0, y: 1},
687 |       {x: 1, y: 1},
688 |       {x: 2, y: 1},
689 |     ],
690 |     L: [
691 |       {x: 2, y: 0},
692 |       {x: 0, y: 1},
693 |       {x: 1, y: 1},
694 |       {x: 2, y: 1},
695 |     ],
696 |   },
697 |   toneClass = (pieceType) => `tone-${pieceType.toLowerCase()}`,
698 |   LINE_CLEAR_LABELS = {1: 'SINGLE', 2: 'DOUBLE', 3: 'TRIPLE', 4: 'QUAD'},
699 |   createCellGrid = (container, count, className) => {

```

```

700 |   const fragment = document.createDocumentFragment(),
701 |   cells = [];
702 |   for (let index = 0; index < count; index += 1) {
703 |     const cell = document.createElement('div');
704 |     ((cell.className = className),
705 |     cell.setAttribute('aria-hidden', 'true'),
706 |     fragment.append(cell),
707 |     cells.push(cell));
708 |   }
709 |   return (container.replaceChildren(fragment), cells);
710 | },
711 | setText = (node, value) => {
712 |   node.textContent === value && (node.textContent = value);
713 | },
714 | formatTimer = (ms, rounding) => {
715 |   const normalizedMs = Math.max(0, ms),
716 |   totalSeconds =
717 |     'ceil' === rounding ? Math.ceil(normalizedMs / 1e3) : Math.floor(normalizedMs / 1e3);
718 |   return `${Math.floor(totalSeconds / 60)}:${(totalSeconds % 60).toString().padStart(2, '0')}`;
719 | },
720 | formatStopwatch = (ms) => {
721 |   const normalizedMs = Math.max(0, Math.floor(ms)),
722 |   totalSeconds = Math.floor(normalizedMs / 1e3),
723 |   millis = normalizedMs % 1e3;
724 |   return `${Math.floor(totalSeconds / 60)}:${(totalSeconds % 60).toString().padStart(2, '0')}.${millis.toString().padStar
  ↪ t(3, '0')}`;
725 | },
726 | formatGravity = (gravityG) => `${gravityG < 1 ? gravityG.toFixed(3) : gravityG.toFixed(2)}G`,
727 | formatMetric = (mode, value) =>
728 |   null === value
729 |     ? '-'
730 |     : 'time' === GAME_MODE_RECORD_METRICS[mode]
731 |       ? formatStopwatch(value)
732 |       : `${value}`,
733 |   initialHandling = () => {
734 |     try {
735 |       const raw = globalThis.localStorage?.getItem('tetris_handling_v1');
736 |       if (!raw) return { ... DEFAULT_HANDLING };
737 |       const parsed = JSON.parse(raw);
738 |       return normalizeHandling(parsed);
739 |     } catch {
740 |       return { ... DEFAULT_HANDLING };
741 |     }
742 |   })();
743 |   let previousStatus = 'ready',
744 |   leaderboardCandidate = null;
745 |   const engine = ((config) => {
746 |     let state = createInitialState(
747 |       config?.random ?? Math.random,
748 |       normalizeHandling(config?.handling ?? {}),
749 |       config?.initialBoard,
750 |       config?.mode ?? 'marathon',
751 |     );
752 |     const fillQueue = (targetLength) => {
753 |       for (; state.queue.length < targetLength; ) {
754 |         0 === state.bag.length && (state.bag = createShuffledBag(state.random));
755 |         const nextType = state.bag.shift();
756 |         if (!nextType) break;
757 |         state.queue.push(nextType);
758 |       }
759 |     },
760 |     spawnFromQueue = () => {
761 |       fillQueue(6);
762 |       const nextType = state.queue.shift();
763 |       if (!nextType) return !1;
764 |       const spawned = createActivePiece(nextType);
765 |       return collides(state.board, spawned)
766 |         ? ((state.active = null), (state.status = 'gameover'), !1)
767 |         : ((state.active = spawned), !0);
768 |     },
769 |     resetForNewRun = () => {

```

```

770 | ((state = createInitialState(state.random, state.handling, config?.initialBoard, state.mode)),
771 | fillQueue(6),
772 | spawnFromQueue(),
773 | (state.status = 'running'));
774 | },
775 | completeRun = () => {
776 | ((state.active = null),
777 | (state.status = 'completed'),
778 | (state.softDropActive = !1),
779 | (state.gravityCarry = 0));
780 | },
781 | maybeResetLockTimer = () => {
782 | state.active &&
783 | (state.active.lockResets >= 15 ||
784 | ((state.active.lockDelayMs = 0), (state.active.lockResets += 1)));
785 | },
786 | wouldLeaveGroundAfterLockResets = (wasGrounded, next) =>
787 | wasGrounded &&
788 | null !== state.active &&
789 | state.active.lockResets >= 15 &&
790 | !isGrounded(state.board, next),
791 | tryShift = (dx, dy, fromPlayerInput) => {
792 | if (!state.active) return !1;
793 | const next = { ...state.active, x: state.active.x + dx, y: state.active.y + dy };
794 | if (collides(state.board, next)) return !1;
795 | const wasGrounded = isGrounded(state.board, state.active);
796 | return !(
797 | (fromPlayerInput && wouldLeaveGroundAfterLockResets(wasGrounded, next)) ||
798 | ((state.active.x = next.x),
799 | (state.active.y = next.y),
800 | (state.active.lastKickIndex = null),
801 | 0 !== dy
802 | ? (state.active.lastAction = 'none')
803 | : fromPlayerInput && (state.active.lastAction = 'move'),
804 | fromPlayerInput && wasGrounded && maybeResetLockTimer(),
805 | 0)
806 | );
807 | },
808 | lockActivePiece = () => {
809 | if (!state.active) return;
810 | const lockedPiece = state.active;
811 | for (const cell of getAbsoluteCells(lockedPiece))
812 | cell.y >= 0 &&
813 | cell.y < 40 &&
814 | cell.x >= 0 &&
815 | cell.x < 10 &&
816 | (state.board[cell.y][cell.x] = lockedPiece.type);
817 | const tspin = ((board, piece) => {
818 | if ('T' !== piece.type || 'rotate' !== piece.lastAction) return 'none';
819 | const pivot_x = piece.x + 1,
820 | pivot_y = piece.y + 1,
821 | corners = [
822 | { x: pivot_x - 1, y: pivot_y - 1 },
823 | { x: pivot_x + 1, y: pivot_y - 1 },
824 | { x: pivot_x - 1, y: pivot_y + 1 },
825 | { x: pivot_x + 1, y: pivot_y + 1 },
826 | ];
827 | if (corners.filter((corner) => isCellBlocked(board, corner.x, corner.y)).length < 3)
828 | return 'none';
829 | const [firstFront, secondFront] = [{ 0: [0, 1], 1: [1, 3], 2: [2, 3], 3: [0, 2] }][
830 | piece.rotation
831 | ];
832 | return 2 ===
833 | [corners[firstFront], corners[secondFront]].filter((corner) =>
834 | isCellBlocked(board, corner.x, corner.y),
835 | ).length
836 | ? 'full'
837 | : 'mini';
838 | })(state.board, lockedPiece),
839 | linesCleared = ((board) => {
840 | const filledRows = [];

```

```

841 |         for (let y = 0; y < 40; y += 1)
842 |             board[y].every((cell) => null === cell) && filledRows.push(y);
843 |         if (0 === filledRows.length) return 0;
844 |         const filledRowSet = new Set(filledRows),
845 |               remainingRows = board.filter((_, rowIndex) => !filledRowSet.has(rowIndex)),
846 |               emptyRows = Array.from({ length: filledRows.length }, () => createEmptyRow());
847 |         return (board.splice(0, 40, ... emptyRows, ... remainingRows), filledRows.length);
848 |     })(state.board),
849 |     perfectClear = state.board.every((row) => row.every((cell) => null === cell));
850 |     linesCleared > 0 ? (state.combo += 1) : (state.combo = -1);
851 |     const b2bBreaks =
852 |         linesCleared > 0 &&
853 |         state.b2bChain > 0 &&
854 |         !keepsBackToBack(linesCleared, tspin, perfectClear),
855 |     attackBreakdown = ((event) => {
856 |         const baseAttack =
857 |             ((lines = event.lines),
858 |             'full' === (tspin = event.tspin)
859 |             ? (TSPIN_ATTACK[lines] ?? 0)
860 |             : 'mini' === tspin
861 |             ? (TSPIN_MINI_ATTACK[lines] ?? 0)
862 |             : (LINE_CLEAR_ATTACK[lines] ?? 0));
863 |         var lines, tspin;
864 |         const comboAttack = ((baseAttack, combo) =>
865 |             baseAttack > 0
866 |             ? Math.floor(baseAttack * (1 + 0.25 * Math.max(combo, 0)))
867 |             : combo >= 2
868 |             ? Math.floor(Math.log(1 + 1.25 * combo))
869 |             : 0)(baseAttack, event.combo),
870 |             difficult = isDifficultClear(event.lines, event.tspin),
871 |             keepsB2B = keepsBackToBack(event.lines, event.tspin, event.perfectClear),
872 |             b2bAttack = difficult && event.b2bActive ? 1 : 0,
873 |             surgeAttack = Math.max(0, Math.floor(event.surgeAttack));
874 |         return {
875 |             baseAttack: baseAttack,
876 |             comboAttack: comboAttack,
877 |             b2bAttack: b2bAttack,
878 |             allClearAttack: 0,
879 |             surgeAttack: surgeAttack,
880 |             total: comboAttack + b2bAttack + 0 + surgeAttack,
881 |             difficult: difficult,
882 |             keepsB2B: keepsB2B,
883 |         };
884 |     })({
885 |         lines: linesCleared,
886 |         tspin: tspin,
887 |         b2bActive: state.b2bChain > 0,
888 |         combo: state.combo,
889 |         perfectClear: perfectClear,
890 |         surgeAttack: b2bBreaks ? state.surgeCharge : 0,
891 |     }),
892 |     scoreBreakdown = ((event) => {
893 |         const level = Math.max(1, Math.floor(event.level)),
894 |             base =
895 |                 ((lines = event.lines),
896 |                 ('full' === (tspin = event.tspin)
897 |                 ? (TSPIN_SCORE[lines] ?? 0)
898 |                 : 'mini' === tspin
899 |                 ? (TSPIN_MINI_SCORE[lines] ?? 0)
900 |                 : (LINE_CLEAR_SCORE[lines] ?? 0)) * level);
901 |         var lines, tspin;
902 |         const difficult = isDifficultClear(event.lines, event.tspin),
903 |             keepsB2B = keepsBackToBack(event.lines, event.tspin, event.perfectClear),
904 |             b2bBonus = difficult && event.b2bActive ? Math.floor(0.5 * base) : 0,
905 |             comboBonus = event.lines > 0 && event.combo > 0 ? 50 * event.combo * level : 0,
906 |             allClearBonus = event.perfectClear && event.lines > 0 ? 3500 * level : 0,
907 |             dropBonus = event.softDropCells + 2 * event.hardDropCells;
908 |         return {
909 |             base: base,
910 |             b2bBonus: b2bBonus,
911 |             comboBonus: comboBonus,

```

```

912 |         allClearBonus: allClearBonus,
913 |         dropBonus: dropBonus,
914 |         total: base + b2bBonus + comboBonus + allClearBonus + dropBonus,
915 |         difficult: difficult,
916 |         keepsB2B: keepsB2B,
917 |     });
918 |     }({
919 |         level: blitzLevelForLines(state.lines),
920 |         lines: linesCleared,
921 |         tspin: tspin,
922 |         b2bActive: state.b2bChain > 0,
923 |         combo: state.combo,
924 |         perfectClear: perfectClear,
925 |         softDropCells: lockedPiece.softDropCells,
926 |         hardDropCells: lockedPiece.hardDropCells,
927 |     });
928 |     var b2bChain;
929 |     ((state.score += scoreBreakdown.total),
930 |     (state.attackSent += attackBreakdown.total),
931 |     linesCleared > 0 &&
932 |     (perfectClear
933 |     ? (state.b2bChain += 2)
934 |     : attackBreakdown.difficult
935 |     ? (state.b2bChain += 1)
936 |     : (state.b2bChain = 0)),
937 |     (state.surgeCharge -= (b2bChain = state.b2bChain) >= 4 ? b2bChain : 0),
938 |     linesCleared > 0 &&
939 |     ((state.clearFeedbackId += 1),
940 |     (state.lastClearFeedback = {
941 |         id: state.clearFeedbackId,
942 |         lines: linesCleared,
943 |         tspin: tspin,
944 |         perfectClear: perfectClear,
945 |         attack: attackBreakdown.total,
946 |         combo: state.combo,
947 |         b2bChain: state.b2bChain,
948 |         difficult: attackBreakdown.difficult,
949 |     })),
950 |     (state.lines += linesCleared),
951 |     (state.canHold = !0),
952 |     (state.gravityCarry = 0),
953 |     'fortyLines' === state.mode && state.lines >= 40 ? completeRun() : spawnFromQueue());
954 |     },
955 |     tryRotate = (direction) => {
956 |         if (!state.active) return !1;
957 |         const wasGrounded = isGrounded(state.board, state.active),
958 |         result = ({
959 |             pieceType: pieceType,
960 |             x: x,
961 |             y: y,
962 |             rotation: rotation,
963 |             direction: direction,
964 |             cellsFor: cellsFor,
965 |             isBlocked: isBlocked,
966 |         }) => {
967 |             const targetRotation = getTargetRotation(rotation, direction),
968 |             targetCells = cellsFor(pieceType, targetRotation),
969 |             kickTests = ((pieceType, from, direction) => {
970 |                 const to = getTargetRotation(from, direction);
971 |                 if ('O' === pieceType) return 0_KICK;
972 |                 const key = ((from, to) => `${from}>${to}`)(from, to);
973 |                 return '180' === direction
974 |                     ? 'I' === pieceType
975 |                     ? (I_180_KICKS[key] ?? 0_KICK)
976 |                     : (JLSTZ_180_KICKS[key] ?? 0_KICK)
977 |                     : 'I' === pieceType
978 |                     ? (I_90_KICKS[key] ?? 0_KICK)
979 |                     : (JLSTZ_90_KICKS[key] ?? 0_KICK);
980 |             })(pieceType, rotation, direction);
981 |             for (let index = 0; index < kickTests.length; index += 1) {
982 |                 const kick = kickTests[index],

```

```

983 |         testX = x + kick.x,
984 |         testY = y + kick.y;
985 |         if (!targetCells.some((cell) => isBlocked(testX + cell.x, testY + cell.y)))
986 |             return {
987 |                 success: !0,
988 |                 x: testX,
989 |                 y: testY,
990 |                 rotation: targetRotation,
991 |                 kickIndex: index,
992 |             };
993 |     }
994 |     return { success: !1, x: x, y: y, rotation: rotation, kickIndex: null };
995 | })(
996 |     pieceType: state.active.type,
997 |     x: state.active.x,
998 |     y: state.active.y,
999 |     rotation: state.active.rotation,
1000 |     direction: direction,
1001 |     cellsFor: pieceCellsFor,
1002 |     isBlocked: (x, y) => isCellBlocked(state.board, x, y),
1003 | );
1004 | if (!result.success) return !1;
1005 | const next = { ...state.active, x: result.x, y: result.y, rotation: result.rotation };
1006 | return (
1007 |     !wouldLeaveGroundAfterLockResets(wasGrounded, next) &&
1008 |     ((state.active.x = result.x),
1009 |     (state.active.y = result.y),
1010 |     (state.active.rotation = result.rotation),
1011 |     (state.active.lastAction = 'rotate'),
1012 |     (state.active.lastKickIndex = result.kickIndex),
1013 |     wasGrounded && maybeResetLockTimer(),
1014 |     !0)
1015 | );
1016 | };
1017 | return (
1018 |     fillQueue(6),
1019 |     {
1020 |         dispatch: (action) =>
1021 |             ? ((state.handling = normalizeHandling(action.payload)), !0)
1022 |             : 'setMode' === action.type
1023 |             ? 'running' !== state.status &&
1024 |             'paused' !== state.status &&
1025 |             (((mode = state.mode) => {
1026 |                 (state = createInitialState(
1027 |                     state.random,
1028 |                     state.handling,
1029 |                     config?.initialBoard,
1030 |                     mode,
1031 |                 )),
1032 |                 fillQueue(6));
1033 |             })(action.payload),
1034 |             !0)
1035 |             : 'restart' === action.type
1036 |             ? (resetForNewRun(), !0)
1037 |             : 'start' === action.type
1038 |             ? 'paused' === state.status
1039 |             ? ((state.status = 'running'), !0)
1040 |             : ('ready' === state.status ||
1041 |             'gameover' === state.status ||
1042 |             'completed' === state.status) &&
1043 |             (resetForNewRun(), !0)
1044 |             : 'togglePause' === action.type
1045 |             ? 'running' === state.status
1046 |             ? ((state.status = 'paused'), !0)
1047 |             : 'paused' === state.status && ((state.status = 'running'), !0)
1048 |             : 'running' === state.status &&
1049 |             ((action) => {
1050 |                 switch (action.type) {
1051 |                     case 'moveLeft':
1052 |                         return tryShift(-1, 0, !0);

```

```

1054 |         case 'moveRight':
1055 |             return tryShift(1, 0, !0);
1056 |         case 'rotateCW':
1057 |             return tryRotate('cw');
1058 |         case 'rotateCCW':
1059 |             return tryRotate('ccw');
1060 |         case 'rotate180':
1061 |             return tryRotate('180');
1062 |         case 'hardDrop':
1063 |             return (() => {
1064 |                 if (!state.active) return !1;
1065 |                 let dropped = 0;
1066 |                 for (; tryShift(0, 1, !1);) dropped += 1;
1067 |                 return (
1068 |                     (state.active.hardDropCells += dropped),
1069 |                     dropped > 0 && (state.active.lastAction = 'none'),
1070 |                     lockActivePiece(),
1071 |                     !0
1072 |                 );
1073 |             })();
1074 |         case 'hold':
1075 |             return (() => {
1076 |                 if (!state.active || !state.canHold) return !1;
1077 |                 const currentType = state.active.type;
1078 |                 if (state.hold) {
1079 |                     const swapType = state.hold;
1080 |                     if (
1081 |                         ((state.hold = currentType),
1082 |                         (state.active = createActivePiece(swapType)),
1083 |                         collides(state.board, state.active))
1084 |                     )
1085 |                         return ((state.active = null), (state.status = 'gameover'), !1);
1086 |                 } else
1087 |                     ((state.hold = currentType), (state.active = null), spawnFromQueue());
1088 |                 return ((state.canHold = !1), (state.gravityCarry = 0), !0);
1089 |             })();
1090 |         default:
1091 |             return !1;
1092 |     }
1093 | })(action),
1094 | tick: (dtMs) => {
1095 |     if ('running' !== state.status || !state.active) return;
1096 |     const nextElapsedMs = state.elapsedMs + dtMs;
1097 |     if (
1098 |         ((state.elapsedMs =
1099 |             'blitz' === state.mode ? Math.min(nextElapsedMs, 12e4) : nextElapsedMs),
1100 |         'blitz' === state.mode && nextElapsedMs > 12e4)
1101 |     )
1102 |         return void completeRun();
1103 |     const gravity = gravityForState(state),
1104 |         effectiveGravity = state.softDropActive
1105 |             ? Math.max(gravity, state.handling.sdfG)
1106 |             : gravity;
1107 |     for (
1108 |         state.gravityCarry += (effectiveGravity * dtMs) / 16.666666666666668;
1109 |         state.gravityCarry >= 1 && state.active;
1110 |     ) {
1111 |         if (!tryShift(0, 1, !1)) {
1112 |             state.gravityCarry = 0;
1113 |             break;
1114 |         }
1115 |         (state.softDropActive && (state.active.softDropCells += 1), (state.gravityCarry -= 1));
1116 |     }
1117 |     state.active &&
1118 |         (isGrounded(state.board, state.active)
1119 |             ? ((state.active.lockDelayMs += dtMs),
1120 |             state.active.lockDelayMs >= 500 && lockActivePiece())
1121 |             : (state.active.lockDelayMs = 0));
1122 | },
1123 | setSoftDropActive: (active) => {
1124 |     state.softDropActive = active;

```



```

1125 |     },
1126 |     getSnapshot: () => {
1127 |         const visibleBoard = state.board.slice(20).map((row) => [... row]),
1128 |         activeCells = state.active
1129 |         ? getAbsoluteCells(state.active)
1130 |         .map(normalizeVisiblePoint)
1131 |         .filter((point) => null !== point)
1132 |         : [],
1133 |         marginElapsedMs = Math.max(0, state.elapsedMs - 120000.00000000001),
1134 |         marginMsRemaining = Math.max(0, 120000.00000000001 - state.elapsedMs),
1135 |         level = blitzLevelForLines(state.lines);
1136 |     return {
1137 |         mode: state.mode,
1138 |         resultMetric: GAME_MODE_RECORD_METRICS[state.mode],
1139 |         status: state.status,
1140 |         boardVisible: visibleBoard,
1141 |         activeCells: activeCells,
1142 |         activeType: state.active?.type ?? null,
1143 |         ghostCells: computeGhostCells(state.board, state.active),
1144 |         hold: state.hold,
1145 |         next: state.queue.slice(0, 5),
1146 |         score: state.score,
1147 |         lines: state.lines,
1148 |         level: level,
1149 |         elapsedMs: state.elapsedMs,
1150 |         timeRemainingMs: 'blitz' === state.mode ? Math.max(0, 12e4 - state.elapsedMs) : null,
1151 |         linesRemaining: 'fortyLines' === state.mode ? Math.max(0, 40 - state.lines) : null,
1152 |         gravityG: gravityForState(state),
1153 |         marginMsRemaining: marginMsRemaining,
1154 |         marginElapsedMs: marginElapsedMs,
1155 |         combo: state.combo,
1156 |         b2bChain: state.b2bChain,
1157 |         lastClearFeedback: state.lastClearFeedback ?? { ... state.lastClearFeedback } : null,
1158 |         handling: { ... state.handling },
1159 |     };
1160 | },
1161 | getHandling: () => ({ ... state.handling }),
1162 | }
1163 | );
1164 | }({ handling: initialHandling });
1165 | let currentMode = engine.getSnapshot().mode,
1166 | bestMetric = loadBestMetric(currentMode),
1167 | leaderboard = loadLeaderboard(currentMode);
1168 | const renderer = (() => {
1169 |     const boardGrid = document.querySelector('#board-grid'),
1170 |     nextGrids = [... document.querySelectorAll('.next-grid')],
1171 |     holdGrid = document.querySelector('#hold-grid'),
1172 |     scoreValue = document.querySelector('#score-value'),
1173 |     highScoreValue = document.querySelector('#high-score-value'),
1174 |     primaryStatLabel = document.querySelector('#primary-stat-label'),
1175 |     bestLabel = document.querySelector('#best-label'),
1176 |     gravityRow = document.querySelector('#stat-gravity-row'),
1177 |     linesRow = document.querySelector('#stat-lines-row'),
1178 |     marginRow = document.querySelector('#stat-margin-row'),
1179 |     gravityLabel = document.querySelector('#gravity-label'),
1180 |     gravityValue = document.querySelector('#gravity-value'),
1181 |     linesLabel = document.querySelector('#lines-label'),
1182 |     linesValue = document.querySelector('#lines-value'),
1183 |     marginLabel = document.querySelector('#margin-label'),
1184 |     marginValue = document.querySelector('#margin-value'),
1185 |     clearToast = document.querySelector('#clear-toast'),
1186 |     comboToast = document.querySelector('#combo-toast'),
1187 |     b2bToast = document.querySelector('#b2b-toast'),
1188 |     perfectClearToast = document.querySelector('#perfect-clear-toast'),
1189 |     leaderboardList = document.querySelector('#leaderboard-list'),
1190 |     leaderboardTitle = document.querySelector('#leaderboard-title'),
1191 |     leaderboardSubtitle = document.querySelector('#leaderboard-subtitle'),
1192 |     leaderboardPrompt = document.querySelector('#leaderboard-prompt'),
1193 |     leaderboardPromptScoreLabel = document.querySelector('#leaderboard-prompt-score-label'),
1194 |     leaderboardPromptScore = document.querySelector('#leaderboard-prompt-score'),
1195 |     leaderboardForm = document.querySelector('#leaderboard-form'),

```

```

1196 |     · · · · ·leaderboardNameInput = document.querySelector('leaderboard-name-input'),
1197 |     · · · · ·modeButtons = [ ... document.querySelectorAll('.mode-option')],
1198 |     · · · · ·startButton = document.querySelector('#start-btn'),
1199 |     · · · · ·pauseButton = document.querySelector('#pause-btn'),
1200 |     · · · · ·restartButton = document.querySelector('#restart-btn'),
1201 |     · · · · ·dasInput = document.querySelector('#das-input'),
1202 |     · · · · ·arrInput = document.querySelector('#arr-input'),
1203 |     · · · · ·sdfInput = document.querySelector('#sdf-input');
1204 |     · · · · ·if (
1205 |     · · · · · · · · !(
1206 |     · · · · · · · · · · boardGrid · &&
1207 |     · · · · · · · · · · 5 === nextGrids.length · &&
1208 |     · · · · · · · · · · holdGrid · &&
1209 |     · · · · · · · · · · scoreValue · &&
1210 |     · · · · · · · · · · highScoreValue · &&
1211 |     · · · · · · · · · · primaryStatLabel · &&
1212 |     · · · · · · · · · · bestLabel · &&
1213 |     · · · · · · · · · · gravityRow · &&
1214 |     · · · · · · · · · · linesRow · &&
1215 |     · · · · · · · · · · marginRow · &&
1216 |     · · · · · · · · · · gravityLabel · &&
1217 |     · · · · · · · · · · gravityValue · &&
1218 |     · · · · · · · · · · linesLabel · &&
1219 |     · · · · · · · · · · linesValue · &&
1220 |     · · · · · · · · · · marginLabel · &&
1221 |     · · · · · · · · · · marginValue · &&
1222 |     · · · · · · · · · · clearToast · &&
1223 |     · · · · · · · · · · comboToast · &&
1224 |     · · · · · · · · · · b2bToast · &&
1225 |     · · · · · · · · · · perfectClearToast · &&
1226 |     · · · · · · · · · · leaderboardList · &&
1227 |     · · · · · · · · · · leaderboardTitle · &&
1228 |     · · · · · · · · · · leaderboardSubtitle · &&
1229 |     · · · · · · · · · · leaderboardPrompt · &&
1230 |     · · · · · · · · · · leaderboardPromptScoreLabel · &&
1231 |     · · · · · · · · · · leaderboardPromptScore · &&
1232 |     · · · · · · · · · · leaderboardForm · &&
1233 |     · · · · · · · · · · leaderboardNameInput · &&
1234 |     · · · · · · · · · · 3 === modeButtons.length · &&
1235 |     · · · · · · · · · · startButton · &&
1236 |     · · · · · · · · · · pauseButton · &&
1237 |     · · · · · · · · · · restartButton · &&
1238 |     · · · · · · · · · · dasInput · &&
1239 |     · · · · · · · · · · arrInput · &&
1240 |     · · · · · · · · · · sdfInput
1241 |     · · · · · )
1242 |     · · · · · )
1243 |     · · · · · throw new Error('Renderer targets are missing');
1244 |     · · · · · const boardCells = createCellGrid(boardGrid, 200, 'cell'),
1245 |     · · · · · nextCells = nextGrids.map((grid) => createCellGrid(grid, 16, 'preview-cell')),
1246 |     · · · · · holdCells = createCellGrid(holdGrid, 16, 'preview-cell');
1247 |     · · · · · ((dasInput.min = `${handlingRanges_dasMs.min}`),
1248 |     · · · · · (dasInput.max = `${handlingRanges_dasMs.max}`),
1249 |     · · · · · (arrInput.min = `${handlingRanges_arrMs.min}`),
1250 |     · · · · · (arrInput.max = `${handlingRanges_arrMs.max}`),
1251 |     · · · · · (sdfInput.min = `${handlingRanges_sdfG.min}`),
1252 |     · · · · · (sdfInput.max = `${handlingRanges_sdfG.max}`));
1253 |     · · · · · const paintPreview = (cells, pieceType) => {
1254 |     · · · · · · · · if (
1255 |     · · · · · · · · · · (cells.forEach((cell) => {
1256 |     · · · · · · · · · · · · cell.className = 'preview-cell';
1257 |     · · · · · · · · · · })),
1258 |     · · · · · · · · · · pieceType)
1259 |     · · · · · · · · )
1260 |     · · · · · · · · for (const point of PREVIEW_CELLS[pieceType]) {
1261 |     · · · · · · · · · · const cell = cells[4 * point.y + point.x];
1262 |     · · · · · · · · · · cell · && (cell.className = `preview-cell ${toneClass(pieceType)}-is-active`);
1263 |     · · · · · · · · · · }
1264 |     · · · · · · · · },
1265 |     · · · · · · · · hideLeaderboardPrompt = () => {
1266 |     · · · · · · · · · · ((leaderboardPrompt.hidden = !0), (leaderboardNameInput.value = ''));

```

```

1267 |     };
1268 |     let prevCombo = -1,
1269 |     prevB2b = 0,
1270 |     prevClearFeedbackId = 0,
1271 |     prevScore = 0,
1272 |     clearToastUntilMs = 0,
1273 |     comboToastUntilMs = 0,
1274 |     b2bToastUntilMs = 0,
1275 |     perfectClearToastUntilMs = 0,
1276 |     comboToastValue = 0,
1277 |     b2bToastValue = 0;
1278 |     const restartAnimation = (element, className) => {
1279 |       (element.classList.remove(className),
1280 |       element.getBoundingClientRect(),
1281 |       element.classList.add(className));
1282 |     };
1283 |     boardGrid.addEventListener('animationend', (event) => {
1284 |       ('b2b-board-shake' === event.animationName && boardGrid.classList.remove('is-b2b-shaking'),
1285 |       'clear-board-pulse' === event.animationName &&
1286 |       boardGrid.classList.remove('is-clear-pulsing', 'is-clear-pulsing-strong'));
1287 |     });
1288 |     const setMiniRow = (row, labelNode, valueNode, label, value, hidden = !1) => {
1289 |       ((row.hidden = hidden), hidden || (setText(labelNode, label), setText(valueNode, value)));
1290 |     };
1291 |     return {
1292 |       render: (snapshot, bestMetric2) => {
1293 |         const now = performance.now(),
1294 |         locked = new Array(200).fill(null),
1295 |         ghostMask = new Array(200).fill(!1),
1296 |         activeMask = new Array(200).fill(!1);
1297 |         for (let y = 0; y < snapshot.boardVisible.length; y += 1)
1298 |           for (let x = 0; x < snapshot.boardVisible[y].length; x += 1)
1299 |             locked[10 * y + x] = snapshot.boardVisible[y][x];
1300 |         for (const point of snapshot.ghostCells) {
1301 |           const index = 10 * point.y + point.x;
1302 |           index >= 0 && index < ghostMask.length && (ghostMask[index] = !0);
1303 |         }
1304 |         for (const point of snapshot.activeCells) {
1305 |           const index = 10 * point.y + point.x;
1306 |           index >= 0 && index < activeMask.length && (activeMask[index] = !0);
1307 |         }
1308 |         for (let index = 0; index < boardCells.length; index += 1) {
1309 |           const cell = boardCells[index];
1310 |           cell.className = 'cell';
1311 |           const lockedPiece = locked[index];
1312 |           (lockedPiece && cell.classList.add(toneClass(lockedPiece), 'is-locked'),
1313 |           lockedPiece || !ghostMask[index] || activeMask[index] || cell.classList.add('is-ghost'),
1314 |           activeMask[index] &&
1315 |           snapshot.activeType &&
1316 |           cell.classList.add(toneClass(snapshot.activeType), 'is-active'));
1317 |         }
1318 |         for (let index = 0; index < nextCells.length; index += 1)
1319 |           paintPreview(nextCells[index], snapshot.next[index] ?? null);
1320 |         if (
1321 |           (paintPreview(holdCells, snapshot.hold),
1322 |           snapshot.score < prevScore && (prevScore = snapshot.score),
1323 |           'score' === snapshot.resultMetric &&
1324 |           snapshot.score > prevScore &&
1325 |           restartAnimation(scoreValue, 'is-score-popping'),
1326 |           ((snapshot) => {
1327 |             const locked = 'running' === snapshot.status || 'paused' === snapshot.status;
1328 |             for (const button of modeButtons) {
1329 |               const active = button.dataset.mode === snapshot.mode;
1330 |               (button.classList.toggle('is-active', active),
1331 |               (button.disabled = locked),
1332 |               button.setAttribute('aria-pressed', active ? 'true' : 'false'));
1333 |             }
1334 |           })(snapshot),
1335 |           ((snapshot, bestMetric2) => {
1336 |             return 'fortyLines' === snapshot.mode
1337 |             ? (setText(primaryStatLabel, 'Time'),

```

```

1338 | .setScore(scoreValue, formatStopwatch(snapshot.elapsedMs)),
1339 | .setBestLabel('Best Time'),
1340 | .setHighScore(highScoreValue, formatMetric(snapshot.mode, bestMetric2)),
1341 | .setMiniRow(
1342 |   gravityRow,
1343 |   gravityLabel,
1344 |   gravityValue,
1345 |   'Lines Left',
1346 |   `${snapshot.linesRemaining ?? 0}`,
1347 | ),
1348 | .setMiniRow(
1349 |   linesRow,
1350 |   linesLabel,
1351 |   linesValue,
1352 |   'Gravity',
1353 |   formatGravity(snapshot.gravityG),
1354 | ),
1355 | void setMiniRow(marginRow, marginLabel, marginValue, 'Margin', '', !0))
1356 | : 'blitz' === snapshot.mode
1357 | ? (setText(primaryStatLabel, 'Score'),
1358 |   setText(scoreValue, `${snapshot.score}`),
1359 |   setText(bestLabel, 'High Score'),
1360 |   setText(highScoreValue, formatMetric(snapshot.mode, bestMetric2)),
1361 |   setMiniRow(
1362 |     gravityRow,
1363 |     gravityLabel,
1364 |     gravityValue,
1365 |     'Time Left',
1366 |     formatTimer(snapshot.timeRemainingMs ?? 0, 'ceil'),
1367 |   ),
1368 |   setMiniRow(linesRow, linesLabel, linesValue, 'Level', `${snapshot.level}`),
1369 |   void setMiniRow(marginRow, marginLabel, marginValue, 'Lines', `${snapshot.lines}`))
1370 | : (setText(primaryStatLabel, 'Score'),
1371 |   setText(scoreValue, `${snapshot.score}`),
1372 |   setText(bestLabel, 'High Score'),
1373 |   setText(highScoreValue, formatMetric(snapshot.mode, bestMetric2)),
1374 |   setMiniRow(
1375 |     gravityRow,
1376 |     gravityLabel,
1377 |     gravityValue,
1378 |     'Gravity',
1379 |     formatGravity(snapshot.gravityG),
1380 |   ),
1381 |   setMiniRow(linesRow, linesLabel, linesValue, 'Lines', `${snapshot.lines}`),
1382 |   void setMiniRow(
1383 |     marginRow,
1384 |     marginLabel,
1385 |     marginValue,
1386 |     'Margin',
1387 |     ((remainingMs = snapshot.marginMsRemaining),
1388 |      (elapsedMs = snapshot.marginElapsedMs),
1389 |      remainingMs > 0
1390 |      ? formatTimer(remainingMs, 'ceil')
1391 |      : `${formatTimer(elapsedMs, 'floor')}`),
1392 |   ));
1393 | var remainingMs, elapsedMs;
1394 | })(snapshot, bestMetric2),
1395 | boardGrid.classList.toggle('is-paused', 'paused' === snapshot.status),
1396 | 'gameover' === snapshot.status ||
1397 | 'completed' === snapshot.status ||
1398 | leaderboardPrompt.hidden ||
1399 | hideLeaderboardPrompt(),
1400 | snapshot.lastClearFeedback)
1401 | ) {
1402 |   if (snapshot.lastClearFeedback.id !== prevClearFeedbackId) {
1403 |     const feedback = snapshot.lastClearFeedback,
1404 |           strongClear = feedback.perfectClear || feedback.difficult || 4 === feedback.lines;
1405 |     (prevClearFeedbackId = feedback.id),
1406 |     feedback.perfectClear
1407 |     ? ((clearToastUntilMs = 0),
1408 |       (perfectClearToastUntilMs = now + 1200),

```

```

1409 |         .setText(perfectClearToast, 'ALL\nCLEAR'),
1410 |         (perfectClearToast.hidden = !1),
1411 |         restartAnimation(perfectClearToast, 'is-showing'))
1412 |         : ((clearToastUntilMs = now + 1200),
1413 |         clearToast.classList.toggle('is-major', strongClear),
1414 |         setText(
1415 |             clearToast,
1416 |             ((feedback) =>
1417 |                 feedback.perfectClear
1418 |                 ? 'PERFECT-CLEAR'
1419 |                 : 'mini' === feedback.tspin
1420 |                 ? `T-SPIN-MINI-${LINE_CLEAR_LABELS[feedback.lines]}`
1421 |                 : 'full' === feedback.tspin
1422 |                 ? `T-SPIN-${LINE_CLEAR_LABELS[feedback.lines]}`
1423 |                 : LINE_CLEAR_LABELS[feedback.lines])(feedback),
1424 |             )),
1425 |         ((feedback) => {
1426 |             if (feedback.attack <= 0) return;
1427 |             const strongClear = feedback.perfectClear || feedback.difficult;
1428 |             (boardGrid.classList.remove('is-clear-pulsing', 'is-clear-pulsing-strong'),
1429 |             boardGrid.style.setProperty(
1430 |                 '--clear-pulse-bg',
1431 |                 ((attack) => {
1432 |                     const intensity = ((value = attack / 10), Math.min(Math.max(value, 0), 1));
1433 |                     var value;
1434 |                     const chroma = 0.12 + 0.08 * intensity,
1435 |                     hue = 205 - 175 * intensity,
1436 |                     alpha = 16 + 16 * intensity;
1437 |                     return `oklch(${(82 - 22 * intensity).toFixed(2)}% ${chroma.toFixed(3)} ${hue.toFixed(1)}deg / ${alpha.
1438 |                         toFixed(2)}%)`);
1439 |                 })(feedback.attack),
1440 |                 )),
1441 |             boardGrid.getBoundingClientRect(),
1442 |             boardGrid.classList.add(
1443 |                 strongClear ? 'is-clear-pulsing-strong' : 'is-clear-pulsing',
1444 |                 )),
1445 |             }
1446 |         } else prevClearFeedbackId = 0;
1447 |         (snapshot.combo > prevCombo &&
1448 |         snapshot.combo > 0 &&
1449 |         ((comboToastValue = snapshot.combo),
1450 |         (comboToastUntilMs = now + 1200),
1451 |         .setText(comboToast, `COMBO x${comboToastValue}`)),
1452 |         snapshot.b2bChain > prevB2b &&
1453 |         snapshot.b2bChain > 0 &&
1454 |         ((b2bToastValue = snapshot.b2bChain),
1455 |         (b2bToastUntilMs = now + 1200),
1456 |         .setText(b2bToast, `B2B x${b2bToastValue}`),
1457 |         boardGrid.classList.remove('is-b2b-shaking'),
1458 |         boardGrid.getBoundingClientRect(),
1459 |         boardGrid.classList.add('is-b2b-shaking')),
1460 |         (clearToast.hidden = now >= clearToastUntilMs),
1461 |         (comboToast.hidden = now >= comboToastUntilMs),
1462 |         (b2bToast.hidden = now >= b2bToastUntilMs),
1463 |         (perfectClearToast.hidden = now >= perfectClearToastUntilMs),
1464 |         perfectClearToast.hidden && perfectClearToast.classList.remove('is-showing'),
1465 |         (prevScore = snapshot.score),
1466 |         (prevCombo = snapshot.combo),
1467 |         (prevB2b = snapshot.b2bChain));
1468 |     },
1469 |     bindControls: (bindings) => {
1470 |         for (const button of modeButtons)
1471 |             button.addEventListener('click', () => {
1472 |                 isGameModeId(button.dataset.mode) && bindings.onModeChange(button.dataset.mode);
1473 |             });
1474 |         (startButton.addEventListener('click', bindings.onStart),
1475 |         pauseButton.addEventListener('click', bindings.onPause),
1476 |         restartButton.addEventListener('click', bindings.onRestart));
1477 |         const onHandlingInput = () => {
1478 |             bindings.onHandlingChange({

```

```

1479 |         dasMs: Number(dasInput.value),
1480 |         arrMs: Number(arrInput.value),
1481 |         sdfG: Number(sdfInput.value),
1482 |     });
1483 | };
1484 |     dasInput.addEventListener('change', onHandlingInput),
1485 |     arrInput.addEventListener('change', onHandlingInput),
1486 |     sdfInput.addEventListener('change', onHandlingInput),
1487 |     leaderboardForm.addEventListener('submit', (event) => {
1488 |         (event.preventDefault(),
1489 |         bindings.onLeaderboardSubmit(leaderboardNameInput.value),
1490 |         hideLeaderboardPrompt());
1491 |     }),
1492 |     leaderboardNameInput.addEventListener('keydown', (event) => {
1493 |         event.stopPropagation();
1494 |     }),
1495 |     leaderboardNameInput.addEventListener('keyup', (event) => {
1496 |         event.stopPropagation();
1497 |     }));
1498 | },
1499 |     setHandlingInputs: (handling) => {
1500 |         ((dasInput.value = `${handling.dasMs}`),
1501 |         (arrInput.value = `${handling.arrMs}`),
1502 |         (sdfInput.value = `${handling.sdfG}`));
1503 |     },
1504 |     renderLeaderboard: (mode, entries) => {
1505 |         const fragment = document.createDocumentFragment();
1506 |         (setText(leaderboardTitle, `${GAME_MODE_LABELS[mode]} Records`),
1507 |         setText(
1508 |             leaderboardSubtitle,
1509 |             'time' === GAME_MODE_RECORD_METRICS[mode] ? 'Best Times' : 'Top 20',
1510 |         ));
1511 |         for (let index = 0; index < 20; index += 1) {
1512 |             const entry = entries[index],
1513 |                 item = document.createElement('li');
1514 |             item.className = entry ? 'leaderboard-item' : 'leaderboard-item-is-empty';
1515 |             const rank = document.createElement('span');
1516 |             ((rank.className = 'leaderboard-rank'), (rank.textContent = `${index + 1}`));
1517 |             const name = document.createElement('span');
1518 |             ((name.className = 'leaderboard-name'), (name.textContent = entry ? name ?? '-' : '-'));
1519 |             const score = document.createElement('span');
1520 |             ((score.className = 'leaderboard-score'),
1521 |             (score.textContent = entry
1522 |                 ? formatMetric(
1523 |                     mode,
1524 |                     'time' === GAME_MODE_RECORD_METRICS[mode] ? entry.elapsedMs : entry.score,
1525 |                 )
1526 |                 : '-'));
1527 |             const detail = document.createElement('span');
1528 |             ((detail.className = 'leaderboard-detail'),
1529 |             (detail.textContent = entry
1530 |                 ? 'time' === GAME_MODE_RECORD_METRICS[mode]
1531 |                 ? `${entry.lines}L · ${entry.score}pts`
1532 |                 : `${entry.lines}L · ${formatStopwatch(entry.elapsedMs)}`
1533 |                 : ''));
1534 |             item.append(rank, name, score, detail),
1535 |             fragment.append(item));
1536 |         }
1537 |         leaderboardList.replaceChildren(fragment);
1538 |     },
1539 |     showLeaderboardPrompt: (candidate) => {
1540 |         (setText(
1541 |             leaderboardPromptScoreLabel,
1542 |             'time' === GAME_MODE_RECORD_METRICS[candidate.mode] ? 'Time' : 'Score',
1543 |         ),
1544 |         setText(
1545 |             leaderboardPromptScore,
1546 |             ((candidate) =>
1547 |                 'time' === GAME_MODE_RECORD_METRICS[candidate.mode]
1548 |                 ? formatStopwatch(candidate.elapsedMs)
1549 |                 : `${candidate.score}`)(candidate),

```

```

1550 |         },
1551 |         (leaderboardNameInput.value = ''),
1552 |         (leaderboardPrompt.hidden = !1),
1553 |         window.requestAnimationFrame(() => {
1554 |             (leaderboardNameInput.focus(), leaderboardNameInput.select());
1555 |         }));
1556 |     },
1557 | };
1558 | })();
1559 | (renderer.setHandlingInputs(initialHandling), renderer.renderLeaderboard(currentMode, leaderboard));
1560 | const loadRecordsForMode = (mode) => {
1561 |     ((currentMode = mode),
1562 |     (bestMetric = loadBestMetric(mode)),
1563 |     (leaderboard = loadLeaderboard(mode)),
1564 |     renderer.renderLeaderboard(mode, leaderboard));
1565 | };
1566 | renderer.bindControls({
1567 |     onStart: () => {
1568 |         ((leaderboardCandidate = null), engine.dispatch({ type: 'start' }));
1569 |     },
1570 |     onPause: () => {
1571 |         engine.dispatch({ type: 'togglePause' });
1572 |     },
1573 |     onRestart: () => {
1574 |         ((leaderboardCandidate = null), engine.dispatch({ type: 'restart' }));
1575 |     },
1576 |     onModeChange: (mode) => {
1577 |         engine.dispatch({ type: 'setMode', payload: mode }) &&
1578 |         ((leaderboardCandidate = null),
1579 |         loadRecordsForMode(mode),
1580 |         renderer.render(engine.getSnapshot(), bestMetric));
1581 |     },
1582 |     onHandlingChange: (nextHandling) => {
1583 |         const normalized = normalizeHandling(nextHandling);
1584 |         (engine.dispatch({ type: 'setHandling', payload: normalized }),
1585 |         renderer.setHandlingInputs(normalized),
1586 |         ((handling) => {
1587 |             try {
1588 |                 const normalized = normalizeHandling(handling);
1589 |                 globalThis.localStorage?.setItem('tetris_handling_v1', JSON.stringify(normalized));
1590 |             } catch {}
1591 |         }))(normalized));
1592 |     },
1593 |     onLeaderboardSubmit: (name) => {
1594 |         if (!leaderboardCandidate) return;
1595 |         const createdAt = Date.now(),
1596 |             entry = {
1597 |                 id: globalThis.crypto?.randomUUID?.() ?? `${Date.now()}-${Math.random()}`,
1598 |                 mode: leaderboardCandidate.mode,
1599 |                 name: name,
1600 |                 score: leaderboardCandidate.score,
1601 |                 lines: leaderboardCandidate.lines,
1602 |                 elapsedMs: leaderboardCandidate.elapsedMs,
1603 |                 createdAt: createdAt,
1604 |             };
1605 |         ((leaderboard = ((mode, entries, entry) =>
1606 |             normalizeLeaderboardEntries(mode, [...leaderboard, { ...entry, mode: mode }]))(
1607 |             leaderboardCandidate.mode,
1608 |             0,
1609 |             entry,
1610 |         )),
1611 |         ((mode, entries) => {
1612 |             const leaderboards = loadLeaderboards();
1613 |             ((leaderboards[mode] = normalizeLeaderboardEntries(mode, entries)),
1614 |             writeJson('tetris_leaderboards_by_mode_v2', leaderboards));
1615 |         })(leaderboardCandidate.mode, leaderboard),
1616 |         renderer.renderLeaderboard(leaderboardCandidate.mode, leaderboard),
1617 |         (leaderboardCandidate = null));
1618 |     },
1619 | });
1620 | const input = ({

```

```

1621 |   ·target: target,
1622 |   ·dispatch: dispatch,
1623 |   ·getHandling: getHandling,
1624 |   ·isRunning: isRunning,
1625 | }) => {
1626 |   ·const pressed = new Set();
1627 |   ·let horizontalOwner = null,
1628 |   ·  dasElapsedMs = 0,
1629 |   ·  arrElapsedMs = 0;
1630 |   ·const resetHorizontalTimers = () => {
1631 |   ·  ·((dasElapsedMs = 0), (arrElapsedMs = 0));
1632 |   ·},
1633 |   ·moveByOwner = () =>
1634 |   ·  ·'left' === horizontalOwner
1635 |   ·  ·? dispatch({ type: 'moveLeft' })
1636 |   ·  ·: 'right' === horizontalOwner && dispatch({ type: 'moveRight' }),
1637 |   ·handoffOwnerIfNeeded = () => {
1638 |   ·  ·const leftPressed = pressed.has('ArrowLeft'),
1639 |   ·  ·rightPressed = pressed.has('ArrowRight');
1640 |   ·  ·if (!leftPressed || !rightPressed)
1641 |   ·  ·  ·return leftPressed
1642 |   ·  ·  ·? ((horizontalOwner = 'left'),
1643 |   ·  ·  ·  ·resetHorizontalTimers(),
1644 |   ·  ·  ·  ·void dispatch({ type: 'moveLeft' }))
1645 |   ·  ·  ·: rightPressed
1646 |   ·  ·  ·? ((horizontalOwner = 'right'),
1647 |   ·  ·  ·  ·resetHorizontalTimers(),
1648 |   ·  ·  ·  ·void dispatch({ type: 'moveRight' }))
1649 |   ·  ·  ·: void (horizontalOwner = null);
1650 |   ·},
1651 |   ·onKeyDown = (event) => {
1652 |   ·  ·if (
1653 |   ·    ·HANDLED_CODES.has(event.code) &&
1654 |   ·    ·(event.preventDefault(), !event.repeat && !pressed.has(event.code))
1655 |   ·  ·)
1656 |   ·  ·switch ((pressed.add(event.code), event.code)) {
1657 |   ·  ·  ·case 'Enter':
1658 |   ·  ·  ·  ·return void dispatch({ type: 'start' });
1659 |   ·  ·  ·case 'KeyP':
1660 |   ·  ·  ·  ·return void dispatch({ type: 'togglePause' });
1661 |   ·  ·  ·case 'KeyR':
1662 |   ·  ·  ·  ·return void dispatch({ type: 'restart' });
1663 |   ·  ·  ·case 'ArrowLeft':
1664 |   ·  ·  ·  ·return (
1665 |   ·  ·  ·    ·(horizontalOwner = 'left'),
1666 |   ·  ·  ·    ·resetHorizontalTimers(),
1667 |   ·  ·  ·    ·void dispatch({ type: 'moveLeft' })
1668 |   ·  ·  ·  ·);
1669 |   ·  ·  ·case 'ArrowRight':
1670 |   ·  ·  ·  ·return (
1671 |   ·  ·  ·    ·(horizontalOwner = 'right'),
1672 |   ·  ·  ·    ·resetHorizontalTimers(),
1673 |   ·  ·  ·    ·void dispatch({ type: 'moveRight' })
1674 |   ·  ·  ·  ·);
1675 |   ·  ·  ·case 'ArrowUp':
1676 |   ·  ·  ·case 'KeyX':
1677 |   ·  ·  ·  ·return void dispatch({ type: 'rotateCW' });
1678 |   ·  ·  ·case 'KeyZ':
1679 |   ·  ·  ·  ·return void dispatch({ type: 'rotateCCW' });
1680 |   ·  ·  ·case 'KeyA':
1681 |   ·  ·  ·  ·return void dispatch({ type: 'rotate180' });
1682 |   ·  ·  ·case 'Space':
1683 |   ·  ·  ·  ·return void dispatch({ type: 'hardDrop' });
1684 |   ·  ·  ·case 'KeyC':
1685 |   ·  ·  ·case 'ShiftLeft':
1686 |   ·  ·  ·case 'ShiftRight':
1687 |   ·  ·  ·  ·return void dispatch({ type: 'hold' });
1688 |   ·  ·  ·default:
1689 |   ·  ·  ·  ·return;
1690 |   ·  ·}
1691 |   ·},

```



```

1692 |     ·onKeyUp·=>·(event)·=>·{
1693 |     ·    ·HANDLED_CODES.has(event.code)·&&
1694 |     ·    ·    ·(pressed.delete(event.code),
1695 |     ·    ·    ·'ArrowLeft'·≠=·event.code·||·'left'·≠=·horizontalOwner
1696 |     ·    ·    ·?·'ArrowRight'·≡=·event.code·&&·'right'·≡=·horizontalOwner·&&·handoffOwnerIfNeeded()
1697 |     ·    ·    ·:·handoffOwnerIfNeeded());
1698 |     ·};
1699 |     ·return·(
1700 |     ·    ·target.addEventListener('keydown',·onKeyDown),
1701 |     ·    ·target.addEventListener('keyup',·onKeyUp),
1702 |     ·    ·{
1703 |     ·        ·update·:(dtMs)·=>·{
1704 |     ·            ·if·(!isRunning()·||·!horizontalOwner)·return;
1705 |     ·            ·const·keyCode·=·'left'·≡=·horizontalOwner?·'ArrowLeft'·:·'ArrowRight';
1706 |     ·            ·if·(!pressed.has(keyCode))·return·void·handoffOwnerIfNeeded();
1707 |     ·            ·const·handling·=·getHandling();
1708 |     ·            ·if·(((dasElapsedMs·+=·dtMs),·!(dasElapsedMs·<·handling.dasMs)))
1709 |     ·            ·    ·if·(0·≠=·handling.arrMs)
1710 |     ·            ·    ·    ·for·(
1711 |     ·            ·    ·    ·    ·arrElapsedMs·+=·dtMs;
1712 |     ·            ·    ·    ·    ·arrElapsedMs·>=·handling.arrMs·&&((arrElapsedMs·-=·handling.arrMs),·moveByOwner());
1713 |     ·            ·    ·    ·);
1714 |     ·            ·else·for·(;·moveByOwner(););
1715 |     ·        ·},
1716 |     ·        ·destroy·:·()·=>·{
1717 |     ·            ·(target.removeEventListener('keydown',·onKeyDown),
1718 |     ·            ·    ·target.removeEventListener('keyup',·onKeyUp));
1719 |     ·        ·},
1720 |     ·        ·isSoftDropActive·:·()·=>·pressed.has('ArrowDown'),
1721 |     ·    ·};
1722 |     ·);
1723 |     ·})({
1724 |     ·target·:·window,
1725 |     ·dispatch·:·(action)·=>·engine.dispatch(action),
1726 |     ·getHandling·:·()·=>·engine.getHandling(),
1727 |     ·isRunning·:·()·=>·'running'·≡=·engine.getSnapshot().status,
1728 |     ·});
1729 |     let·lastTimestamp·=·performance.now();
1730 |     const·loop·=·(timestamp)·=>·{
1731 |     ·    ·const·dtMs·=·Math.min(timestamp·-·lastTimestamp,·50);
1732 |     ·    ·((lastTimestamp·=·timestamp),
1733 |     ·    ·    ·input.update(dtMs),
1734 |     ·    ·    ·engine.setSoftDropActive(input.isSoftDropActive()),
1735 |     ·    ·    ·engine.tick(dtMs));
1736 |     ·    ·const·snapshot·=·engine.getSnapshot(),
1737 |     ·    ·    ·justEnded·=·!(
1738 |     ·    ·    ·    ·('gameover'·≠=·snapshot.status·&&·'completed'·≠=·snapshot.status)·||
1739 |     ·    ·    ·    ·('running'·≠=·previousStatus·&&·'paused'·≠=·previousStatus)
1740 |     ·    ·    ·),
1741 |     ·    ·    ·nextBestMetric·=·((snapshot)·=>
1742 |     ·    ·    ·    ·'fortyLines'·≡=·snapshot.mode
1743 |     ·    ·    ·    ·?·'completed'·≡=·snapshot.status
1744 |     ·    ·    ·    ·?·snapshot.elapsedMs
1745 |     ·    ·    ·    ·:·null
1746 |     ·    ·    ·:·snapshot.score)(snapshot);
1747 |     ·    ·if·(
1748 |     ·    ·    ·(null·≠=·nextBestMetric·&&
1749 |     ·    ·    ·    ·((mode,·value,·current)·=>·{
1750 |     ·    ·    ·    ·    ·const·normalized·=·normalizeMetricForMode(mode,·value);
1751 |     ·    ·    ·    ·    ·return·(
1752 |     ·    ·    ·    ·    ·    ·null·≠=·normalized·&&
1753 |     ·    ·    ·    ·    ·    ·(null·≡=·current·||
1754 |     ·    ·    ·    ·    ·    ·    ·('time'·≡=·GAME_MODE_RECORD_METRICS[mode]
1755 |     ·    ·    ·    ·    ·    ·    ·?·normalized·<·current
1756 |     ·    ·    ·    ·    ·    ·    ·:·normalized·>·current))
1757 |     ·    ·    ·    ·);
1758 |     ·    ·    ·})(snapshot.mode,·nextBestMetric,·bestMetric)·&&
1759 |     ·    ·    ·((bestMetric·=·nextBestMetric),
1760 |     ·    ·    ·    ·((mode,·value)·=>·{
1761 |     ·    ·    ·    ·    ·const·normalized·=·normalizeMetricForMode(mode,·value);
1762 |     ·    ·    ·    ·    ·if·(null·≡=·normalized)·return;

```

```

1763 |         const metrics = loadBestMetrics();
1764 |         ((metrics[mode] = normalized), writeJson('tetris_best_records_by_mode_v2', metrics));
1765 |     })(snapshot.mode, nextBestMetric),
1766 |     justEnded)
1767 |   ) {
1768 |     const candidate = ((snapshot) =>
1769 |       'fortyLines' === snapshot.mode
1770 |       ? 'completed' !== snapshot.status || snapshot.lines < 40
1771 |       ? null
1772 |       : {
1773 |         mode: snapshot.mode,
1774 |         score: snapshot.score,
1775 |         lines: snapshot.lines,
1776 |         elapsedMs: snapshot.elapsedMs,
1777 |       }
1778 |       : 'marathon' === snapshot.mode && 'gameover' !== snapshot.status
1779 |       ? null
1780 |       : {
1781 |         mode: snapshot.mode,
1782 |         score: snapshot.score,
1783 |         lines: snapshot.lines,
1784 |         elapsedMs: snapshot.elapsedMs,
1785 |       })(snapshot);
1786 |     candidate &&
1787 |     ((mode, entries, candidate) => {
1788 |       const normalizedScore = normalizeInteger(candidate.score, 0),
1789 |       normalizedLines = normalizeInteger(candidate.lines, 0),
1790 |       normalizedElapsedMs = normalizeInteger(candidate.elapsedMs, 0);
1791 |       if (null === normalizedScore || null === normalizedLines || null === normalizedElapsedMs)
1792 |         return !1;
1793 |       const candidateEntry = {
1794 |         id: 'candidate',
1795 |         mode: mode,
1796 |         name: 'PLAYER',
1797 |         score: normalizedScore,
1798 |         lines: normalizedLines,
1799 |         elapsedMs: normalizedElapsedMs,
1800 |         createdAt: Number.MAX_SAFE_INTEGER,
1801 |       };
1802 |       if ('time' === GAME_MODE_RECORD_METRICS[mode] && candidateEntry.elapsedMs <= 0) return !1;
1803 |       const normalizedEntries = normalizeLeaderboardEntries(mode, entries);
1804 |       return (
1805 |         normalizedEntries.length < 20 ||
1806 |         compareLeaderboardEntries(mode)(candidateEntry, normalizedEntries[19]) < 0
1807 |       );
1808 |     })(snapshot.mode, leaderboard, candidate)
1809 |     ? ((leaderboardCandidate = candidate), renderer.showLeaderboardPrompt(candidate))
1810 |     : (leaderboardCandidate = null);
1811 |   } else
1812 |     'gameover' !== snapshot.status &&
1813 |     'completed' !== snapshot.status &&
1814 |     (leaderboardCandidate = null);
1815 |   (snapshot.mode !== currentMode && loadRecordsForMode(snapshot.mode),
1816 |   renderer.render(snapshot, bestMetric),
1817 |   (previousStatus = snapshot.status),
1818 |   window.requestAnimationFrame(loop));
1819 | };
1820 | (renderer.render(engine.getSnapshot(), bestMetric),
1821 | window.requestAnimationFrame(loop),
1822 | window.addEventListener('beforeunload', () => {
1823 |   input.destroy();
1824 | })));
1825 |

```

==== assets/index-Cb95KwZL.css (14408 B / 2a57b7ea8268) ====

```

1 | :root {
2 |   --bg: oklch(100% 0 0deg);
3 |   --surface: oklch(98% 0 0deg);
4 |   --line: oklch(30% 0 0deg);
5 |   --line-soft: oklch(85% 0 0deg);
6 |   --text: oklch(20% 0 0deg);

```

```

7 | --muted: oklch(45% 0 0deg);
8 | --ghost: oklch(75% 0 0deg / 75%);
9 | --tone-i: oklch(82% 0.15 220deg);
10 | --tone-o: oklch(90% 0.15 98deg);
11 | --tone-t: oklch(70% 0.15 315deg);
12 | --tone-s: oklch(82% 0.15 150deg);
13 | --tone-z: oklch(72% 0.15 25deg);
14 | --tone-j: oklch(66% 0.15 255deg);
15 | --tone-l: oklch(78% 0.15 70deg);
16 | --effect-clear-border: oklch(62% 0.14 205deg);
17 | --effect-clear-overlay: oklch(82% 0.12 205deg / 16%);
18 | --effect-clear-ring: oklch(70% 0.16 205deg / 72%);
19 | --effect-major-border: oklch(67% 0.16 92deg);
20 | --effect-major-bg: oklch(97% 0.05 92deg);
21 | --effect-major-overlay: oklch(88% 0.18 92deg / 22%);
22 | --effect-major-ring: oklch(76% 0.18 92deg / 86%);
23 | --effect-perfect-fill: oklch(100% 0 0deg);
24 | --effect-perfect-stroke: oklch(52% 0.16 88deg);
25 | --radius: 0.5rem;
26 | --gap: 0.75rem;
27 | --font: 'Segoe UI', 'Noto Sans KR', 'Apple SD Gothic Neo', 'Malgun Gothic', sans-serif;
28 | }
29 | *,
30 | *:before,
31 | *:after {
32 |   box-sizing: border-box;
33 | }
34 | * {
35 |   margin: 0;
36 | }
37 | html,
38 | body {
39 |   min-height: 100%;
40 | }
41 | body {
42 |   line-height: 1.5;
43 |   -webkit-font-smoothing: antialiased;
44 | }
45 | img,
46 | picture,
47 | svg,
48 | video {
49 |   display: block;
50 |   max-width: 100%;
51 | }
52 | input,
53 | button,
54 | textarea,
55 | select {
56 |   font: inherit;
57 | }
58 | ul,
59 | ol {
60 |   list-style: none;
61 |   padding: 0;
62 | }
63 | body {
64 |   background: var(--bg);
65 |   color: var(--text);
66 |   font-family: var(--font);
67 | }
68 | #app {
69 |   min-height: 100vh;
70 |   padding: 1rem;
71 |   display: grid;
72 |   gap: var(--gap);
73 |   place-items: center;
74 | }
75 | .layout {
76 |   width: min(72rem, 100%);
77 |   min-height: min(96vh, 60rem);

```

```

78 |   display: grid;
79 |   grid-template-columns: 18rem minmax(0, 26rem) 18rem;
80 |   column-gap: 0.45rem;
81 |   row-gap: var(--gap);
82 |   justify-content: center;
83 |   align-items: start;
84 | }
85 | .panel-card {
86 |   border: 1px solid var(--line);
87 |   border-radius: var(--radius);
88 |   background: var(--surface);
89 | }
90 | .board-anchor {
91 |   position: relative;
92 |   width: 100%;
93 |   max-width: 26rem;
94 |   margin: auto;
95 |   grid-column: 2;
96 |   grid-row: 1;
97 | }
98 | .tetris-board {
99 |   width: 100%;
100 |   aspect-ratio: 10/20;
101 |   margin: 0;
102 |   position: relative;
103 |   border: 1px solid var(--line);
104 |   border-radius: 0.3rem;
105 |   padding: 0.2rem;
106 |   display: grid;
107 |   grid-template-columns: repeat(10, minmax(0, 1fr));
108 |   grid-template-rows: repeat(20, minmax(0, 1fr));
109 |   gap: 0;
110 |   line-height: 0;
111 |   overflow: hidden;
112 | }
113 | .tetris-board:before {
114 |   content: '';
115 |   position: absolute;
116 |   inset: 0;
117 |   border: 0.12rem solid transparent;
118 |   border-radius: inherit;
119 |   pointer-events: none;
120 |   z-index: 1;
121 | }
122 | .tetris-board.is-b2b-shaking {
123 |   animation: b2b-board-shake 0.18s ease-out;
124 |   will-change: transform;
125 | }
126 | .tetris-board.is-clear-pulsing:before,
127 | .tetris-board.is-clear-pulsing-strong:before {
128 |   --clear-pulse-line: var(--effect-clear-ring);
129 |   animation: clear-board-pulse 0.26s ease-out;
130 |   background: var(--clear-pulse-bg, var(--effect-clear-overlay));
131 |   border-color: var(--clear-pulse-line);
132 | }
133 | .tetris-board.is-clear-pulsing-strong:before {
134 |   --clear-pulse-line: var(--effect-major-ring);
135 | }
136 | .tetris-board.is-paused:after {
137 |   content: 'PAUSED';
138 |   position: absolute;
139 |   inset: 0;
140 |   display: grid;
141 |   place-items: center;
142 |   font-size: 1.5rem;
143 |   font-weight: 800;
144 |   letter-spacing: 0.08em;
145 |   color: var(--text);
146 |   text-shadow: 0 0 0.4rem oklch(100% 0 0deg / 0.7);
147 |   pointer-events: none;
148 |   z-index: 2;

```

```

149 | }
150 | @keyframes b2b-board-shake {
151 |   0% {
152 |     transform: translateZ(0);
153 |   }
154 |   18% {
155 |     transform: translate3d(0, -4px, 0);
156 |   }
157 |   36% {
158 |     transform: translate3d(0, 4px, 0);
159 |   }
160 |   54% {
161 |     transform: translate3d(0, -3px, 0);
162 |   }
163 |   72% {
164 |     transform: translate3d(0, 2px, 0);
165 |   }
166 |   to {
167 |     transform: translateZ(0);
168 |   }
169 | }
170 | @keyframes clear-board-pulse {
171 |   0% {
172 |     opacity: 0;
173 |     box-shadow: inset 0 0 #fff0;
174 |   }
175 |   28% {
176 |     opacity: 1;
177 |     box-shadow: inset 0 0 0.16rem var(--clear-pulse-line);
178 |   }
179 |   to {
180 |     opacity: 0;
181 |     box-shadow: inset 0 0 0.45rem #fff0;
182 |   }
183 | }
184 | .cell,
185 | .preview-cell {
186 |   margin: 0;
187 |   border: 0;
188 |   padding: 0;
189 |   display: block;
190 |   width: 100%;
191 |   height: 100%;
192 |   background: var(--surface);
193 | }
194 | .cell.is-ghost {
195 |   background: var(--ghost);
196 | }
197 | .tone-i {
198 |   background: var(--tone-i);
199 | }
200 | .tone-o {
201 |   background: var(--tone-o);
202 | }
203 | .tone-t {
204 |   background: var(--tone-t);
205 | }
206 | .tone-s {
207 |   background: var(--tone-s);
208 | }
209 | .tone-z {
210 |   background: var(--tone-z);
211 | }
212 | .tone-j {
213 |   background: var(--tone-j);
214 | }
215 | .tone-l {
216 |   background: var(--tone-l);
217 | }
218 | .button-row {
219 |   display: flex;

```

```

220 |     gap: 0.35rem;
221 |     flex-wrap: wrap;
222 |     grid-column: 2;
223 |     grid-row: 2;
224 |     justify-content: center;
225 | }
226 | button,
227 | input {
228 |     border: 1px solid var(--line);
229 |     border-radius: 0.35rem;
230 |     padding: 0.35rem 0.5rem;
231 |     background: var(--surface);
232 |     color: var(--text);
233 | }
234 | button {
235 |     cursor: pointer;
236 | }
237 | .side-panel {
238 |     display: grid;
239 |     grid-template-rows: repeat(3, minmax(0, auto));
240 |     gap: var(--gap);
241 | }
242 | .side-panel-left {
243 |     justify-items: stretch;
244 |     grid-column: 1;
245 |     grid-row: 1 / span 2;
246 | }
247 | .side-panel-right {
248 |     justify-items: stretch;
249 |     grid-column: 3;
250 |     grid-row: 1 / span 2;
251 |     border: 1px solid var(--line);
252 |     border-radius: var(--radius);
253 |     background: var(--surface);
254 |     padding: 0.55rem;
255 | }
256 | .panel-card {
257 |     padding: 0.55rem;
258 |     display: grid;
259 |     gap: 0.35rem;
260 |     align-content: start;
261 | }
262 | .panel-card h2 {
263 |     font-size: 0.9rem;
264 | }
265 | .stats-hud {
266 |     display: grid;
267 |     gap: 0.2rem;
268 |     align-content: start;
269 | }
270 | .stats-hud-title {
271 |     font-size: 0.82rem;
272 |     color: var(--muted);
273 | }
274 | .stats-hud-score {
275 |     font-size: 2rem;
276 |     font-weight: 800;
277 |     line-height: 1;
278 | }
279 | #score-value {
280 |     display: inline-block;
281 |     transform-origin: left center;
282 | }
283 | #score-value.is-score-popping {
284 |     animation: score-pop 0.18s ease-out;
285 | }
286 | @keyframes score-pop {
287 |     0% {
288 |         transform: translateY(0) scale(1);
289 |     }
290 |     45% {

```

```

291 | | | transform: translateY(-0.08rem) scale(1.08);
292 | | }
293 | | to {
294 | | | transform: translateY(0) scale(1);
295 | | }
296 | }
297 | .stats-hud-high-score {
298 | | font-size: 0.84rem;
299 | | color: var(--muted);
300 | | display: flex;
301 | | justify-content: space-between;
302 | | gap: 0.4rem;
303 | }
304 | .stats-divider {
305 | | border-top: 1px solid var(--line);
306 | | margin: 0.24rem 0 0.18rem;
307 | }
308 | .stats-mini-list {
309 | | display: grid;
310 | | gap: 0.2rem;
311 | }
312 | .stats-mini-item {
313 | | font-size: 0.82rem;
314 | | color: var(--muted);
315 | | display: flex;
316 | | justify-content: space-between;
317 | | gap: 0.4rem;
318 | }
319 | .preview-panel {
320 | | display: grid;
321 | | gap: 0.35rem;
322 | | align-content: start;
323 | }
324 | .preview-panel h2 {
325 | | font-size: 0.9rem;
326 | }
327 | .controls-help {
328 | | min-width: 0;
329 | | border-top: 1px solid var(--line-soft);
330 | | margin-top: 0.1rem;
331 | | padding-top: 0.55rem;
332 | | display: grid;
333 | | gap: 0.4rem;
334 | | align-content: start;
335 | }
336 | .controls-help h2 {
337 | | font-size: 0.9rem;
338 | }
339 | .controls-help-list {
340 | | margin: 0;
341 | | display: grid;
342 | | gap: 0.28rem;
343 | }
344 | .controls-help-row {
345 | | min-width: 0;
346 | | display: grid;
347 | | grid-template-columns: minmax(5.8rem, 1fr) minmax(0, auto);
348 | | gap: 0.4rem;
349 | | align-items: center;
350 | }
351 | .controls-help-row dt {
352 | | min-width: 0;
353 | | color: var(--muted);
354 | | font-size: 0.78rem;
355 | | line-height: 1.2;
356 | }
357 | .controls-help-row dd {
358 | | min-width: 0;
359 | | margin: 0;
360 | | display: flex;
361 | | flex-wrap: wrap;

```

```

362 | justify-content: flex-end;
363 | gap: 0.18rem;
364 | }
365 | .controls-help-kbd {
366 | min-width: 1.55rem;
367 | border: 1px solid var(--line);
368 | border-bottom-width: 0.16rem;
369 | border-radius: 0.28rem;
370 | padding: 0.08rem 0.28rem 0.1rem;
371 | background: var(--surface);
372 | color: var(--text);
373 | font: inherit;
374 | font-size: 0.72rem;
375 | font-weight: 800;
376 | line-height: 1.15;
377 | text-align: center;
378 | white-space: nowrap;
379 | }
380 | .settings-panel-label {
381 | font-size: 0.82rem;
382 | color: var(--muted);
383 | }
384 | .mode-panel-h2 {
385 | font-size: 0.9rem;
386 | }
387 | .mode-toggle {
388 | display: grid;
389 | grid-template-columns: repeat(3, minmax(0, 1fr));
390 | gap: 0;
391 | }
392 | .mode-option {
393 | min-width: 0;
394 | border-radius: 0;
395 | padding: 0.38rem 0.32rem;
396 | overflow-wrap: anywhere;
397 | font-size: 0.78rem;
398 | font-weight: 800;
399 | line-height: 1.1;
400 | }
401 | .mode-option:first-child {
402 | border-radius: 0.35rem 0 0 0.35rem;
403 | }
404 | .mode-option:last-child {
405 | border-radius: 0 0.35rem 0.35rem 0;
406 | }
407 | .mode-option + .mode-option {
408 | margin-left: -1px;
409 | }
410 | .mode-option.is-active {
411 | background: var(--text);
412 | color: var(--surface);
413 | }
414 | .mode-option.disabled {
415 | cursor: not-allowed;
416 | opacity: 0.55;
417 | }
418 | #event-toast {
419 | position: absolute;
420 | left: 0;
421 | bottom: 0;
422 | transform: translate(calc(-100% - 0.35rem));
423 | z-index: 30;
424 | display: grid;
425 | gap: 0.28rem;
426 | pointer-events: none;
427 | }
428 | .event-toast-item {
429 | margin: 0;
430 | padding: 0.2rem 0.45rem;
431 | border: 1px solid var(--line);
432 | border-radius: 0.3rem;

```



```

433 | background: var(--surface);
434 | color: var(--text);
435 | font-size: 1rem;
436 | font-weight: 800;
437 | letter-spacing: 0.04em;
438 | }
439 | .clear-toast-item {
440 |   border-color: var(--effect-clear-border);
441 | }
442 | .clear-toast-item.is-major {
443 |   border-color: var(--effect-major-border);
444 |   background: var(--effect-major-bg);
445 | }
446 | .perfect-clear-toast {
447 |   position: absolute;
448 |   left: 50%;
449 |   top: 50%;
450 |   z-index: 40;
451 |   width: max-content;
452 |   max-width: calc(100% - 0.5rem);
453 |   transform: translate(-50%, -50%) scale(1);
454 |   pointer-events: none;
455 |   color: var(--effect-perfect-fill);
456 |   font-size: clamp(1.8rem, 12vw, 3.25rem);
457 |   font-weight: 900;
458 |   line-height: 0.86;
459 |   letter-spacing: 0;
460 |   text-align: center;
461 |   text-transform: uppercase;
462 |   white-space: pre-line;
463 |   text-shadow: none;
464 |   -webkit-text-stroke: 0.028em var(--effect-perfect-stroke);
465 |   text-stroke: 0.028em var(--effect-perfect-stroke);
466 | }
467 | .perfect-clear-toast.is-showing {
468 |   animation: perfect-clear-shout 0.76s cubic-bezier(0.16, 1, 0.3, 1) both;
469 | }
470 | .leaderboard-prompt {
471 |   position: absolute;
472 |   inset: 0;
473 |   z-index: 60;
474 |   padding: 1rem;
475 |   display: grid;
476 |   place-items: center;
477 |   background: #ffffffb8;
478 | }
479 | .leaderboard-prompt[hidden] {
480 |   display: none;
481 | }
482 | .leaderboard-form {
483 |   width: min(100%, 18rem);
484 |   border: 1px solid var(--line);
485 |   border-radius: var(--radius);
486 |   padding: 0.75rem;
487 |   display: grid;
488 |   gap: 0.45rem;
489 |   background: var(--surface);
490 |   box-shadow: 0 0.5rem 1.5rem #00000029;
491 | }
492 | .leaderboard-form h2 {
493 |   font-size: 1.15rem;
494 |   line-height: 1.1;
495 | }
496 | .leaderboard-prompt-kicker,
497 | .leaderboard-prompt-score,
498 | .leaderboard-form label {
499 |   color: var(--muted);
500 |   font-size: 0.82rem;
501 | }
502 | .leaderboard-prompt-score span {
503 |   color: var(--text);

```

```

504 |   font-weight: 800;
505 | }
506 | .leaderboard-submit-row {
507 |   display: grid;
508 |   grid-template-columns: minmax(0, 1fr) auto;
509 |   gap: 0.35rem;
510 | }
511 | .leaderboard-submit-row input {
512 |   min-width: 0;
513 |   text-transform: uppercase;
514 | }
515 | .leaderboard-dock {
516 |   width: min(72rem, 100%);
517 | }
518 | .leaderboard-panel {
519 |   border: 1px solid var(--line);
520 |   border-radius: var(--radius);
521 |   padding: 0.55rem;
522 |   display: grid;
523 |   gap: 0.45rem;
524 |   background: var(--surface);
525 | }
526 | .leaderboard-header {
527 |   display: flex;
528 |   align-items: baseline;
529 |   justify-content: space-between;
530 |   gap: 0.5rem;
531 | }
532 | .leaderboard-header h2 {
533 |   font-size: 0.95rem;
534 | }
535 | .leaderboard-header p {
536 |   margin: 0;
537 |   color: var(--muted);
538 |   font-size: 0.78rem;
539 | }
540 | .leaderboard-list {
541 |   margin: 0;
542 |   padding: 0;
543 |   display: grid;
544 |   grid-template-columns: repeat(auto-fit, minmax(13rem, 1fr));
545 |   gap: 0.16rem 0.75rem;
546 |   list-style: none;
547 | }
548 | .leaderboard-item {
549 |   min-width: 0;
550 |   min-height: 2rem;
551 |   border-bottom: 1px solid var(--line-soft);
552 |   padding: 0.16rem 0;
553 |   display: grid;
554 |   grid-template-columns: 2ch minmax(0, 1fr) auto;
555 |   grid-template-areas: 'rank name score' 'rank detail detail';
556 |   column-gap: 0.45rem;
557 |   align-items: center;
558 | }
559 | .leaderboard-item.is-empty {
560 |   color: var(--muted);
561 | }
562 | .leaderboard-rank {
563 |   grid-area: rank;
564 |   color: var(--muted);
565 |   font-size: 0.78rem;
566 |   font-variant-numeric: tabular-nums;
567 |   text-align: right;
568 | }
569 | .leaderboard-name {
570 |   grid-area: name;
571 |   min-width: 0;
572 |   overflow: hidden;
573 |   font-size: 0.86rem;
574 |   font-weight: 800;

```

```

575 |   text-overflow: ellipsis;
576 |   white-space: nowrap;
577 | }
578 | .leaderboard-score {
579 |   grid-area: score;
580 |   font-size: 0.86rem;
581 |   font-weight: 800;
582 |   font-variant-numeric: tabular-nums;
583 | }
584 | .leaderboard-detail {
585 |   grid-area: detail;
586 |   color: var(--muted);
587 |   font-size: 0.72rem;
588 | }
589 | @keyframes perfect-clear-shout {
590 |   0% {
591 |     opacity: 0;
592 |     transform: translate(-50%, -50%) scale(0.96);
593 |   }
594 |   12% {
595 |     opacity: 1;
596 |     transform: translate(-50%, -50%) scale(1.06);
597 |   }
598 |   62% {
599 |     opacity: 1;
600 |     transform: translate(-50%, -50%) scale(1);
601 |   }
602 |   to {
603 |     opacity: 0;
604 |     transform: translate(-50%, -50%) scale(0.98);
605 |   }
606 | }
607 | .preview-grid {
608 |   width: 7rem;
609 |   aspect-ratio: 1/1;
610 |   border: 1px solid var(--line);
611 |   border-radius: 0.3rem;
612 |   padding: 0.2rem;
613 |   display: grid;
614 |   grid-template-columns: repeat(4, minmax(0, 1fr));
615 |   grid-template-rows: repeat(4, minmax(0, 1fr));
616 |   gap: 0;
617 |   line-height: 0;
618 |   overflow: hidden;
619 | }
620 | .next-list {
621 |   display: grid;
622 |   gap: 0.28rem;
623 |   justify-items: start;
624 | }
625 | .next-grid {
626 |   width: 4.8rem;
627 | }
628 | .next-grid,
629 | #hold-grid {
630 |   border: 0;
631 |   background: #bebebe;
632 | }
633 | .next-grid .preview-cell:not(.is-active),
634 | #hold-grid .preview-cell:not(.is-active) {
635 |   background: #bebebe;
636 | }
637 | @media (max-width: 980px) {
638 |   .layout {
639 |     min-height: auto;
640 |     grid-template-columns: 1fr;
641 |   }
642 |   .tetris-board,
643 |   .board-anchor,
644 |   .button-row,
645 |   .side-panel-left,

```

```

646 | .side-panel-right-{
647 |   grid-column: auto;
648 |   grid-row: auto;
649 | }
650 | .side-panel-{
651 |   grid-template-columns: 1fr;
652 |   grid-template-rows: none;
653 | }
654 | #event-toast-{
655 |   left: 50%;
656 |   bottom: 0.45rem;
657 |   width: calc(100% - 1rem);
658 |   transform: translate(-50%);
659 |   justify-items: center;
660 | }
661 | .event-toast-item-{
662 |   max-width: 100%;
663 |   text-align: center;
664 | }
665 | }
666 | @media (min-width: 1380px){
667 |   #app-{
668 |     grid-template-columns: minmax(0, 1fr) clamp(18rem, 22vw, 22rem);
669 |     align-items: stretch;
670 |     justify-items: stretch;
671 |     column-gap: 1rem;
672 |     padding: 0 0 0 1rem;
673 |   }
674 |   .layout-{
675 |     align-self: center;
676 |     justify-self: center;
677 |   }
678 |   .leaderboard-dock-{
679 |     width: 100%;
680 |     min-height: 100vh;
681 |     align-self: stretch;
682 |   }
683 |   .leaderboard-panel-{
684 |     height: 100vh;
685 |     border-width: 0 0 0 1px;
686 |     border-radius: 0;
687 |     padding: 1rem;
688 |     overflow: auto;
689 |   }
690 |   .leaderboard-list-{
691 |     grid-template-columns: 1fr;
692 |   }
693 | }
694 | @media (max-width: 640px){
695 |   .side-panel-{
696 |     grid-template-columns: 1fr;
697 |   }
698 |   .controls-help-row-{
699 |     grid-template-columns: minmax(0, 1fr);
700 |     gap: 0.18rem;
701 |   }
702 |   .controls-help-row-dd-{
703 |     justify-content: flex-start;
704 |   }
705 |   .stats-hud-score-{
706 |     font-size: 1.7rem;
707 |   }
708 |   .event-toast-item-{
709 |     font-size: 0.9rem;
710 |   }
711 | }
712 | @media (prefers-reduced-motion: reduce){
713 |   .tetris-board.is-b2b-shaking,
714 |   .tetris-board.is-clear-pulsing:before,
715 |   .tetris-board.is-clear-pulsing-strong:before,
716 |   .perfect-clear-toast.is-showing,

```

```

717 |     #score-value.is-score-popping-{
718 |     .animation: none;
719 |     will-change: auto;
720 |   }
721 | }
722 |

```

==== index.html (7534 B / bc726d2a0eb1) ====

```

1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>Tetris</title>
7 |     <script type="module" crossorigin src="/assets/index-BM6zjEMh.js"></script>
8 |     <link rel="stylesheet" crossorigin href="/assets/index-Cb95KwZL.css" />
9 |   </head>
10 |   <body>
11 |     <div id="app">
12 |       <main class="layout">
13 |         <aside class="side-panel side-panel-left" aria-label="왼쪽 정보 패널">
14 |           <section class="stats-hud" aria-label="점수 패널">
15 |             <p id="primary-stat-label" class="stats-hud-title">Score</p>
16 |             <p class="stats-hud-score"><span id="score-value">0</span></p>
17 |             <p class="stats-hud-high-score">
18 |               <span id="best-label">High Score</span> <span id="high-score-value">0</span>
19 |             </p>
20 |             <div class="stats-divider" aria-hidden="true"></div>
21 |             <div class="stats-mini-list">
22 |               <p id="stat-gravity-row" class="stats-mini-item">
23 |                 <span id="gravity-label">Gravity</span>
24 |                 <span id="gravity-value">0.020G</span>
25 |               </p>
26 |               <p id="stat-lines-row" class="stats-mini-item">
27 |                 <span id="lines-label">Lines</span>
28 |                 <span id="lines-value">0</span>
29 |               </p>
30 |               <p id="stat-margin-row" class="stats-mini-item">
31 |                 <span id="margin-label">Margin</span>
32 |                 <span id="margin-value">2:00</span>
33 |               </p>
34 |             </div>
35 |           </section>
36 |
37 |           <section class="panel-card settings-panel" aria-label="핸들링 설정">
38 |             <h2>Handling</h2>
39 |             <label for="das-input">DAS (ms)</label>
40 |             <input id="das-input" type="number" step="1" />
41 |
42 |             <label for="arr-input">ARR (ms)</label>
43 |             <input id="arr-input" type="number" step="1" />
44 |
45 |             <label for="sdf-input">SDF (G)</label>
46 |             <input id="sdf-input" type="number" step="0.5" />
47 |           </section>
48 |
49 |           <section class="panel-card mode-panel" aria-label="게임 모드">
50 |             <h2>Mode</h2>
51 |             <div id="mode-toggle" class="mode-toggle" role="group" aria-label="게임 모드 선택">
52 |               <button class="mode-option" type="button" data-mode="marathon">Marathon</button>
53 |               <button class="mode-option" type="button" data-mode="fortyLines">40 Lines</button>
54 |               <button class="mode-option" type="button" data-mode="blitz">BLITZ</button>
55 |             </div>
56 |           </section>
57 |         </aside>
58 |
59 |         <div class="board-anchor">
60 |           <div
61 |             id="board-grid"
62 |             class="tetris-board"
63 |             role="img"

```

```

64 |         .aria-label="10열·20행·게임·그리드"
65 |     ></div>
66 |     <div id="event-toast" aria-live="polite" aria-atomic="true">
67 |         <p id="clear-toast" class="event-toast-item clear-toast-item" hidden></p>
68 |         <p id="combo-toast" class="event-toast-item" hidden></p>
69 |         <p id="b2b-toast" class="event-toast-item" hidden></p>
70 |     </div>
71 |     <p
72 |         id="perfect-clear-toast"
73 |         class="perfect-clear-toast"
74 |         aria-live="polite"
75 |         aria-atomic="true"
76 |         hidden
77 |     ></p>
78 |     <section
79 |         id="leaderboard-prompt"
80 |         class="leaderboard-prompt"
81 |         aria-label="리더보드·이름·입력"
82 |         hidden
83 |     >
84 |         <form id="leaderboard-form" class="leaderboard-form">
85 |             <p class="leaderboard-prompt-kicker">Top·20</p>
86 |             <h2>New·Record</h2>
87 |             <p class="leaderboard-prompt-score">
88 |                 <span id="leaderboard-prompt-score-label">Score</span>
89 |                 <span id="leaderboard-prompt-score">0</span>
90 |             </p>
91 |             <label for="leaderboard-name-input">Name</label>
92 |             <div class="leaderboard-submit-row">
93 |                 <input
94 |                     id="leaderboard-name-input"
95 |                     type="text"
96 |                     maxlength="12"
97 |                     autocomplete="off"
98 |                     inputmode="text"
99 |                     placeholder="PLAYER"
100 |                 />
101 |                 <button type="submit">Save</button>
102 |             </div>
103 |         </form>
104 |     </section>
105 | </div>
106 | <div class="button-row">
107 |     <button id="start-btn" type="button">Start</button>
108 |     <button id="pause-btn" type="button">Pause</button>
109 |     <button id="restart-btn" type="button">Restart</button>
110 | </div>
111 |
112 | <aside class="side-panel side-panel-right" aria-label="오른쪽·미리보기·패널">
113 |     <section class="preview-panel" aria-label="다음·블록·패널">
114 |         <h2>Next</h2>
115 |         <div id="next-list" class="next-list">
116 |             <div class="preview-grid next-grid" data-next-index="0"></div>
117 |             <div class="preview-grid next-grid" data-next-index="1"></div>
118 |             <div class="preview-grid next-grid" data-next-index="2"></div>
119 |             <div class="preview-grid next-grid" data-next-index="3"></div>
120 |             <div class="preview-grid next-grid" data-next-index="4"></div>
121 |         </div>
122 |     </section>
123 |
124 |     <section class="preview-panel" aria-label="홀드·블록·패널">
125 |         <h2>Hold</h2>
126 |         <div id="hold-grid" class="preview-grid"></div>
127 |     </section>
128 |
129 |     <section class="controls-help" aria-label="조작키·안내">
130 |         <h2>조작키</h2>
131 |         <dl class="controls-help-list">
132 |             <div class="controls-help-row">
133 |                 <dt>이동</dt>
134 |                 <dd><kbd>←</kbd><kbd>→</kbd></dd>

```

```

135 |         </div>
136 |     <div class="controls-help-row">
137 |         <dt>소프트 드롭</dt>
138 |         <dd><kbd>↓</kbd></dd>
139 |     </div>
140 |     <div class="controls-help-row">
141 |         <dt>하드 드롭</dt>
142 |         <dd><kbd>Space</kbd></dd>
143 |     </div>
144 |     <div class="controls-help-row">
145 |         <dt>시계 회전</dt>
146 |         <dd><kbd>↑</kbd><kbd>X</kbd></dd>
147 |     </div>
148 |     <div class="controls-help-row">
149 |         <dt>반시계 회전</dt>
150 |         <dd><kbd>Z</kbd></dd>
151 |     </div>
152 |     <div class="controls-help-row">
153 |         <dt>180도 회전</dt>
154 |         <dd><kbd>A</kbd></dd>
155 |     </div>
156 |     <div class="controls-help-row">
157 |         <dt>홀드</dt>
158 |         <dd><kbd>C</kbd><kbd>Shift</kbd></dd>
159 |     </div>
160 |     <div class="controls-help-row">
161 |         <dt>시작</dt>
162 |         <dd><kbd>Enter</kbd></dd>
163 |     </div>
164 |     <div class="controls-help-row">
165 |         <dt>일시정지</dt>
166 |         <dd><kbd>P</kbd></dd>
167 |     </div>
168 |     <div class="controls-help-row">
169 |         <dt>다시 시작</dt>
170 |         <dd><kbd>R</kbd></dd>
171 |     </div>
172 | </dl>
173 | </section>
174 | </aside>
175 | </main>
176 |
177 |     <aside class="leaderboard-dock" aria-label="리더보드">
178 |         <section class="leaderboard-panel">
179 |             <header class="leaderboard-header">
180 |                 <h2 id="leaderboard-title">Leaderboard</h2>
181 |                 <p id="leaderboard-subtitle">Top 20</p>
182 |             </header>
183 |             <ol id="leaderboard-list" class="leaderboard-list"></ol>
184 |         </section>
185 |     </aside>
186 | </div>
187 | </body>
188 | </html>
189 |

```